

Zou_lab_control 主手册

notebook-first + PyQt GUI + remote FPGA pulse-streamer 系统总览

架构、session/devices/timing 分层、sequencer 生命周期、真实硬件
runbook 与 N-slot 扫描模型

2026-06-30

Contents

1	写给新读者	1
1.1	这套系统是做什么的	1
1.2	三台角色的物理图景	1
2	原理：分层与状态归属	3
2.1	neutral_atom 的分层	3
2.2	连接实验	3
3	PulseSequence 与 PulseTableState	5
3.1	两个模型的分工	5
3.2	从状态到边沿表	5
4	Sequencer 生命周期	7
4.1	prepare / fire / wait_done / safe_state	7
4.2	Vivado 持久会话与重复 prepare 缓存	7
4.3	API 等价写法	8
5	真实硬件 Runbook	9
5.1	安装	9
5.2	在 FPGA 电脑 build 和 program	9
5.3	启动 server	10
5.4	控制电脑 notebook	10
6	读出测量：扫描读出时长与一键测温	11
6.1	读出时长 → 保真度	11
6.2	一键测温:release-recapture	11
7	N-slot 扫描模型	13
7.1	为什么是“命名 slot”	13
7.2	scan_table: N_points x N_slots	13
7.3	主机如何编译扫描	14
7.4	端到端的可运行示例	14
7.5	扫描 DAC 值（硬件无缝）	15
7.6	固定延时如何与扫描共存（事件调度的逐信号延时）	15

7.7	扫描 + 采集：一次完整循环	16
7.8	解绑与单点求值	16
8	分层与状态归属（深入）	17
8.1	为什么要分层：一句话的设计哲学	17
8.2	六个子包：职责与状态归属	17
8.2.1	devices/ ——硬件适配器与契约	18
8.2.2	timing/ ——时间真相	18
8.2.3	operations/ ——纯算法（离线）	18
8.2.4	subsystems/ ——实验级编排	18
8.2.5	views/ ——给前端的绘图适配器	19
8.2.6	core/ ——校准与结果模型	19
8.3	na.connect 与会话对象	19
8.3.1	会话拥有哪些状态	19
8.3.2	一个最小会话的全貌	20
8.4	三种后端，同一张会话脸	20
8.5	走通一次真实实验：完整生命周期	20
8.5.1	一段可运行的最小实验	21
8.6	铁律：把边界焊死的不变量	21
8.7	把分层装进脑子里：一张归属速查表	22
9	PulseSequence 与 PulseTableState（深入）	23
9.1	为什么是两个模型	23
9.2	一行边沿表是什么意思	24
9.3	从 PulseSequence 到边沿表：compile_runtime_program	24
9.4	PulseTableState 的状态与 API	26
9.5	一个完整的 PulseTableState 编译实例	27
9.5.1	编出来之后怎么跑实验	28
10	Sequencer 生命周期与远程会话（深入）	29
10.1	四个动作与两台计算机	29
10.2	状态归属：谁持有什么	30
10.3	prepare：准备但不发射	30
10.4	fire：一个显式的 start 脉冲，时序所有权交接	31
10.5	wait_done：只对有限序列轮询	31
10.6	safe_state：回到安全空闲（Stop Pulse 走这条）	32
10.7	常驻 Vivado 会话：慢操作只做一次	32
10.8	流式回填：扫描点比常驻窗口多时怎么办	32
10.9	RemoteSequencer 与 RPyC：客户端代理	33
10.10	五个 Sequencer 类各司其职	33
10.11	repeat_forever 的两种用法：示波器 vs. 相机	34
10.12	从零跑一次真实实验：完整清单	34

11	N-slot 扫描模型与硬件无缝扫描 (深入)	36
11.1	为什么是“命名 slot”而不是 x/y	36
11.2	绑定字段: bind_field 与 scan dot	36
11.2.1	解绑与重新编号	37
11.3	scan_table: N_points × N_slots 的数据矩阵	38
11.4	主机如何编译: 一个仿射模板 + 一张流式点表	38
11.5	DAC 值 + 时长: 同时扫两者 (并叠加一个固定延时)	39
11.6	限制与编译器拒绝的情形	41
12	真实硬件 Runbook (深入)	42
12.1	系统全景: 两台计算机、四个层次	42
12.2	第一步: 在两台机器上安装	43
12.3	第二步 (FPGA 机): 编译 + 烧写	43
12.4	相机成像的物理输出 (必须记住的四根线)	44
12.5	第三步 (FPGA 机): 启动脉冲流服务器	45
12.6	第四步 (控制机): 从 notebook 连上去	46
12.7	排错手册 (按现象查)	47
13	Pulse API: 脚本控制与延时校准	48
13.1	延时校准完整示例	48
14	修改指南	50
14.1	我想改默认时钟	50
14.2	我想加一条新 channel 或新预设	50
14.3	我想加一个扫描参数	50
14.4	我想改文档	50
15	常见问题	51
15.1	脉冲长度是设定的两倍	51
15.2	On Pulse 没报错但示波器无信号	51
15.3	GUI 只显示少数 channel	51
15.4	扫描不通过	51

1

写给新读者

1.1 这套系统是做什么的

`Zou_lab_control` 是一套中性原子实验控制代码库，采用 notebook 优先的工作方式。它把硬件控制、时序、图像分析、绘图和结果对象分层，使得同一份实验逻辑可以在三种后端上运行：

- virtual 设备：完全离线，用于教学和回归测试；
- 真实 qCMOS 相机；
- 远程 FPGA pulse-streamer（脉冲序列器）。

整套系统由三块组成，本手册是**总览手册**，对应另外两本专题手册：

main (本手册) 系统架构、neutral_atom 的 session/devices/timing 分层、`PulseSequence` 与 `PulseTableState` 的区别、sequencer 生命周期、真实硬件 runbook，以及全新的 N-slot 扫描模型。

frontend GUI 与绘图：`qt_fluent` 控件库、脉冲编辑器三个标签页、按字段扫描点 workflow、绘图 API 与 PDF 渲染。见 docs/frontend_manual/。

fpga pulse-streamer 硬件：Artix-7 35T 上的 edge-table RTL、1-tick FIFO 预取流水线、2-bank 流式扫描窗口、N-slot 仿射扫描引擎、模拟总线 DAC 引擎、JTAG-to-AXI 上传流程与资源预算。见 docs/fpga_manual/。

device 设备与实验：设备配置/加载、相机采集、相机读出原理 (sitemap/thresholds/detect)、标定与结果对象、完整实验流程。见 docs/device_manual/。

阅读约定

本手册用中性教学口吻。每个组件归属哪一层、状态住在哪里、调用后应得到什么结果，都会写清楚。维护者用的架构不变式和反模式放在 docs/MAINTAINER_NOTES.md，不放进本手册。

1.2 三台角色的物理图景

真实实验是双电脑结构：一台控制/qCMOS 电脑，一台 FPGA/Vivado 电脑，两者用 RPyC 连接。

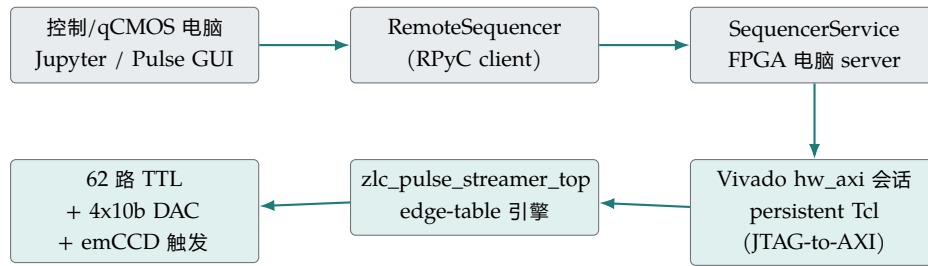


Figure 1.1: 真实硬件路径。GUI/notebook 只生成脉冲状态；编译、上传和 fire 由 FPGA 电脑负责。

GUI 只是脉冲前端。它可以只显示 `trap/cooling/probe/emCCD` 四路，也可以显示全部 XDC 推断出来的 channel；上传时永远按 server 的完整硬件 channel order 编译，未显示或未配置的 channel 补成 off。

2

原理：分层与状态归属

2.1 neutral_atom 的分层

`Zou_lab_control.neutral_atom` 围绕清晰的边界组织：

devices/ 硬件适配器与设备契约。设备只负责硬件动作。包含 `axi_session.py` (JTAG-to-AXI 上传 `VivadoAxiStreamerSession`)、`fpga_pulse_streamer.py` (XDC 推断 + 程序校验)、`sequencer.py`、`sequencer_server.py`。

timing/ 时序的真相来源。`sequence.py` 里的 `PulseSequence`，`pulse_table.py` 里的 `PulseTableState` (GUI 编译模型)，`verilog.py` 的边沿表生成。

operations/ 纯图像/标定/检测算法，可离线运行。

subsystems/ 实验级工作流，例如 `exp.readout`、`exp.timing`。

views/ 接到 frontend 的绘图适配器。

一个关键不变式

相机 capture 只显示**原始**图像。site 圆圈、阈值这些标定叠加属于 readout 结果，不属于相机设备。如果 capture 上出现了 site 圆圈，那是把标定状态错放到了相机层。

2.2 连接实验

`na.connect` 返回一个 `NeutralAtomSession`。三种典型连法：

Python

```
import Zou_lab_control.neutral_atom as na

# 完全离线：虚拟相机 + 虚拟 sequencer
exp = na.connect("virtual")

# 真实硬件：控制电脑连 FPGA 电脑上的 server
exp = na.connect(
```

```
    "remote_template",  
    sequencer={"host": "192.168.0.20", "port": 18861},  
    open_devices=True,  
)
```

session 暴露子系统: `exp.camera` (采图)、`exp.timing` (脉冲/触发)、`exp.readout` (标定与占据检测)、`exp.devices` (底层设备, 例如 `exp.devices.sequencer`)。

3

PulseSequence 与 PulseTableState

3.1 两个模型的分工

时序有两个层次的对象，不要混淆：

PulseSequence 底层、与时间相关的真相来源。它知道每条 channel 在每个时刻的电平，能编译成边沿表（绝对 tick + 完整输出 mask）。notebook、PyQt、远程 FPGA 都共享这同一个真相来源。

PulseTableState GUI 的**编译模型**。它保留用户能理解的概念：一排**周期 (period)**，每个 period 有一个持续时间和一整套数字状态向量；再加上 channel 名称/标签、可见性、延时、模拟总线计划，以及扫描 slot。它本身不拥有硬件，只 `compile()` 成 **PulseSequence**/runtime program。

一句话：**PulseTableState** 是给人编辑的表格，**PulseSequence** 是给时钟执行的边沿。

3.2 从状态到边沿表

Python

```
state =
↳ na.PulseTableState.load("pulses/camera_imaging_address_switch.json")

program = state.compile(
    clock_hz=exp.devices.sequencer.clock_hz,          # 默认 50 MHz
    repeat_forever=False,
)
print(program.ticks)    # 例如 [0, 100000, 105000, 106000, 1105000, 1106000]
print(program.masks)    # 每个 tick 对应的完整输出 mask
```

一行边沿的含义是：**在这个绝对 FPGA tick, 把所有输出切换成这个 mask**。bit 0 对应 `channels[0]`，bit 1 对应 `channels[1]`，以此类推。多条 channel 在同一物理时刻变化，会合并成一条带完整 mask 的边沿。

Camera 预设 50 MHz 的编译结果

```
ticks: 0, 100000, 105000, 106000, 1105000, 1106000  
masks: 513, 512, 2568, 520, 512, 0
```

513 = ch00 + ch09 (cooling + trap), 2568 = ch03 + ch09 + ch11 (probe + trap + emCCD)。相机外触发 bit 是 ch11/emCCD。

4

Sequencer 生命周期

4.1 prepare / fire / wait_done / safe_state

控制电脑通过 `RemoteSequencer` 调用四个动作。它们经 RPyC 到达 FPGA 电脑上的 `SequencerService`，再到 `VivadoAxisStreamerSession`：

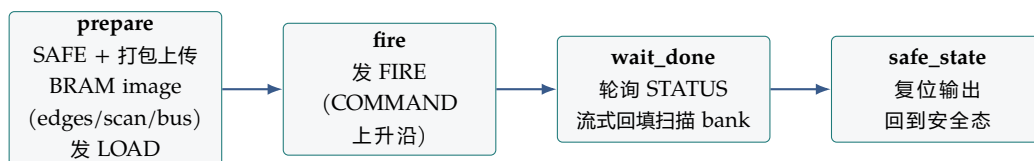


Figure 4.1: 一次 shot 的生命周期。prepare 只是把表写好并校验，不启动；只有 fire 的同步上升沿才是 start 事件。

prepare(program) 先发 SAFE，把编译好的程序打包成 BRAM image (边沿表、重复元数据、扫描点、总线段)，经 JTAG-to-AXI 写进 FPGA，arm 扫描 bank，然后发 LOAD (COMMAND 上升沿，等 `STATUS_LOADED`)。prepare 不启动脉冲，它只是让 FPGA 持有一张已校验的表。

fire() 发 FIRE 命令 (COMMAND 字先清 0 再置位，制造干净的上升沿)。FPGA 只把同步后的上升沿当作 start。这之后，微秒级时序由 FPGA 时钟拥有；Python、RPyC、Windows 调度、Vivado、JTAG 延迟都不再决定 shot 内的边沿。

wait_done() 轮询 `STATUS`，等待有限序列跑完；对超出常驻窗口的流式扫描，它会跟随 `CURSOR` 把空出来的扫描 bank 回填下一段。

safe_state() 发 SAFE，把输出拉回安全态 (`Stop Pulse` 走这条路)。

4.2 Vivado 持久会话与重复 prepare 缓存

`jtag-axi` 后端只启动一次 Vivado Tcl 进程，打开 hardware target，加载一次 `.ltx` (让 Vivado 在已烧录的比特流里找到 `jtag_axi` 核)，然后整个 server 生命周期内复用这个 `hw_axi` 会话来做 prepare/fire/wait_done/safe_state。慢的 hardware-target 建立发生在 server 启动时，而不是第一次 `On Pulse`。

`SequencerService.prepare` 还做了一层缓存：如果新程序的 `sequence_id` (程序内容的哈希) 和已准备的完全相同，就跳过整段上传，直接复用已经载入的程序。是否启用由 `ZLC_SEQUENCER_CACHE_PREPARED` 控制。所以一次扫描里若某点和上一点编译结果一致，第二次 `On Pulse` 只是几次控制写。

4.3 API 等价写法

Python

```
state =  
↳ na.PulseTableState.load("pulses/camera_imaging_address_switch.json")  
program = state.compile(  
    clock_hz=exp.devices.sequencer.clock_hz,  
    repeat_forever=False,  
)  
exp.devices.sequencer.prepare(program)  
exp.devices.sequencer.fire()  
exp.devices.sequencer.wait_done()
```

`repeat_forever=True` 适合示波器找线，输出会一直循环。真实相机采集有限帧时，应让 `readout/camera helper` 先 `arm` 相机，再生成刚好需要帧数的有限触发序列，不要让 `qCMOS` 等一个没有帧数边界的自由循环脉冲。

5

真实硬件 Runbook

5.1 安装

在两台电脑上都装 Python 包; Vivado 只装在 FPGA 电脑。

PowerShell

```
cd D:\ZLC
install_requirements.bat
```

`install_requirements.bat` 把当前解释器记到 `.zlc_python_path`; 之后 `pulse_gui.bat`、`fpga\run_server.bat` 优先用同一个解释器。若 Vivado 不在默认路径:

PowerShell

```
$env:ZLC_PS_VIVADO_BIN = "C:\Xilinx\Vivado\2019.1\bin\vivado.bat"
```

5.2 在 FPGA 电脑 build 和 program

PowerShell

```
cd D:\ZLC
.\fpga\build_and_program.bat --check      # 不连板的 HDL 综合 + 容量自查
.\fpga\build_and_program.bat              # build + program
↪ (create_project.tcl -> program_fpga.tcl)
```

默认工程在 `fpga\build\ps` (短名 `ps` 让 Vivado 深层 `run/.Xil` 临时路径不超过 Windows `MAX_PATH`, 构建仍留在仓库内), `bit/ltx` (`zlc_pulse_streamer_top.bit/.ltx`) 在其 `ps.runs\textbackslash impl_1` 目录下。默认 XDC 是 `address-switch` 板的引脚图 (`fpga\textbackslash board_config\textbackslash board.xdc`, 见该目录 README; 用 `$env:ZLC_PS_XDC` 覆盖)。相机成像预设的关键物理输出:

Camera-imaging subset

```
ch09 -> trap      -> M17
ch00 -> cooling    -> F15
ch03 -> probe     -> N15
ch11 -> emCCD     -> M13    (qCMOS 外触发)
```

默认时钟 50 MHz，最小步进 20 ns。如果示波器看到脉冲长度是设定值的两倍，先检查 control/server/GUI 是否还在沿用旧的 100 MHz 假设。

5.3 启动 server

PowerShell

```
.\fpga\run_server.bat --check-config    # 打印
↪ project/bit/ltx/XDC/channel/trigger/clock/容量
.\fpga\run_server.bat                  # host 0.0.0.0, port 18861
```

5.4 控制电脑 notebook

Python

```
exp = na.connect("remote_template",
                 sequencer={"host": "192.168.0.20", "port": 18861},
                 open_devices=True)

exp.timing.configure_imaging(exposure=2e-3, load=True)
exp.timing.preflight().raise_if_failed()

capture    = exp.camera.capture(frames=1)
sitemap    = exp.readout.sitemap(frames=20, grid_shape=(5, 7))
threshold  = exp.readout.thresholds(frames=120, site=0)
shot       = exp.readout.detect()
```

6

读出测量: 扫描读出时长与一键测温

标定(sitemap / thresholds)之后, `exp.readout` 还提供两类**扫描测量**: 扫描读出时长得保真度曲线, 以及 release-recapture (弹道重捕) **测温**。两者都是同一台**通用扫描引擎** (`operations/measurement.py`) 的实例——扫一个被绑定的脉冲参数、每点采几帧、把帧约简成一个数做成 live 曲线——只换扫描轴 / 每点采帧方式 / 约简器。它们只走相机 / sequencer / 标定契约, 所以**虚拟与真机同一代码路径** (换真机只改 `na.connect`)。

单一真相源

notebook 一行 (`exp.readout.temperature(...)`) 与 Task 控制台 GUI 的一键 Start **调同一个建器** (`build\_temperature\_scan / build\_detection\_scan`), 所以 API 与 GUI 永远不漂移。GUI 路径见 frontend 手册“Task 控制台”章。

6.1 读出时长 → 保真度

`exp.readout.readout\_duration\_fidelity(times, *, shots=60, site=None, ...)` (语义别名 `detection\_time`) 扫一组成像/探测时长 `times` (秒), 每个时长对 ROI 计数分布做 Otsu 阈值 + 双高斯保真度估计, 返回随时长增长的单发保真度曲线。默认 `live=True` (前端定时刷新, `scan.stop()` 提前停)。

6.2 一键测温: release-recapture

物理: 两次成像之间把 trap 关掉 `t\_off`, 原子按热速度自由飞; 再开 trap, 问还有多少原子近到能被重捕。越热飞得越远, 存活随 `t\_off` 衰减越快——**survival-vs-t_off 曲线编码了温度**。曲线只定 $r_c/\sqrt{T}$, 所以**必须**从阱几何给出捕获半径 `capture\_radius` (米) 才能定出 T 。

Python

```
import numpy as np
import Zou_lab_control.neutral_atom as na

exp = na.connect("virtual", sitemap={"grid_shape": (5, 7)}) # 真机换成对应
↪ config
```

```
exp.readout.sitemap(method="box"); exp.readout.thresholds()

# 6 周期双触发序列:trap_off 周期的 duration 绑到 scan slot s0(= t_off)
state = na.build_release_recapture_pulse(channels=list(exp.devices.sequen_
↳ cer.channels))
pulse = na.bind_pulse(exp.devices.sequencer, state)

# 扫 t_off, 每点 2 帧 (成像 1/成像 2), 按 calibration.detect 算逐点存活率
res = exp.readout.temperature(np.linspace(0, 300e-6, 13), pulse=pulse,
↳ shots=16)

# 纯后处理拟合温度 (capture_radius 必填, 米)
fit = na.fit_temperature(res.x, res.y, capture_radius=6e-6)
print(fit.summary()) # {'temperature_uK': ~44..50, 'capture_radius_m':
↳ 6e-06, ...}
```

涉及的 API:

exp.readout.temperature(t_off_s, *, pulse, shots=1, per_site=False, live=True, ...)

扫 t_off_s (秒), 每点经 ReleaseRecapturePlan 采两帧、SurvivalReducer 出存活, 返回 ScanResult (res.x=t_off、res.y= 存活)。需要已定阈值的标定 (存活是二值占据比较)。per_site=True 返回逐站存活。

na.build_release_recapture_pulse(*, channels, exposure=20e-3, trap_off_nominal=20e-6, ...)

建 6 周期 PulseTableState (两相机触发夹一段 trap-off), trap-off 周期的 duration 经 bind_field 绑到 scan slot s0=t_off (与扫描读出时长同一种可扫量)。**必须是**单条双触发序列、不能重复单帧序列 (两帧要同一次装载、中间夹 trap=0)。

na.fit_temperature(t_off, survival, *, capture_radius, mass=RB87_MASS, fit_baseline=True)

纯后处理 (不进采集循环): 把弹道重捕模型拟合到存活曲线, 回 TemperatureFit (temperature_uK 等)。capture_radius 是已知输入 (量级简并), 只拟合 T (与可选 baseline)。

底层组件 operations.temperature 还导出 ReleaseRecapturePlan(两帧 plan)、SurvivalReducer

(逐点存活, 标量或逐站)、release_recapture_survival (物理模型); 以及 exp.readout.build_temperature_scan(...) / build_detection_scan(...) (单一真相源建器, 返回未跑的 ScannedMeasurement)、exp.readout.measurement_specs() (给 GUI 的测量目录)。

测量目录是自动发现的 measurement_specs() 不是写死的 list: 它由 operations.measurement_registry

装配——operations/measurements/ 包里每个 @measurement 工厂 (内置温度/读出时长就住这里) + notebook 里 na.register_measurement(工厂) 的。**新写一个测量丢进那个包、或 na.register_measurement 一下, 就自动进目录、出现在 task_console 的 Add Panel 里**, 无需改任何 list。工厂签名 build(readout) → MeasurementSpec(用 readout.session 拿 exp、复用建器); 相关公开符号: na.measurement(装饰器)、na.register_measurement/na.unregister_measurement、na.axis_range_tuple。

7

N-slot 扫描模型

7.1 为什么是“命名 slot”

这是唯一的扫描概念：被扫的量由你绑定的**命名 slot** (`s0/s1/.....`) 决定，**没有 x/y 二变量的说法**。

任何**逐字段**的值都可以**绑定到一个命名 slot**：

- 一个 period 的持续时间 (`kind="duration"`);
- 一个 period 内某条模拟总线的 DAC 值 (`kind="dac"`)。

只有这**两类**字段可以扫描：`SCAN_SLOT_KINDS = ("duration", "dac")`。**channel 延时不是扫描量**——它是每条 channel 的一个**固定值**（见下文“固定延时如何与扫描共存”），`state.bind_field("delay", ...)` 会直接抛 `ValueError("delay is a fixed per-channel value and cannot be scanned")`。

slot 按绑定顺序命名为 `s0`, `s1`, ...。在 GUI 里点字段旁边的扫描圆点，或在 API 里调用 `state.bind_field(kind, target)` 完成绑定。绑定后该字段的值在表达式里变成 `s0` 这样的符号。

7.2 scan_table: N_points x N_slots

扫描数据是一张 `scan_table`：行 = 一个扫描点，列 `j = slot s\{j\}` 在该 slot 显示单位下的取值（时间 slot 用 ns，`dac` slot 用整数 DAC code）。它可以从 `.npy/.csv/.txt` 载入，或在 GUI 的 Scan 标签页用 Python 现场构造。

point	s0 (ns)	s1 (ns)
0	1000	0
1	2000	500
2	4000	1000

主机编译成
一张仿射边沿模板
+ 一张流式扫描点表；
硬件无缝迭代每个点。

Figure 7.1: 一张两 slot、三点的 `scan_table`。

7.3 主机如何编译扫描

绑定过的 duration 是**时间 slot**，它参与仿射 tick 公式。主机把每行边沿编译成一个 base tick 加 NUM_SLOTS 个定点系数，FPGA 在运行时计算：

$$\text{effective_tick} = \text{base_tick} + \left(\sum_j \text{coeff}_j \cdot \text{slot}_j \right) \gg \text{COEFF_FRAC_BITS} \quad (7.1)$$

其中 COEFF_FRAC_BITS=8 (定点小数位)。这样一个很大的一维或多维扫描**不会**让边沿表变长——边沿数限制的是模板复杂度，不是扫描点数。

时间表达式只能是 slot 的**仿射**函数 (数字、s0...、加减乘除、括号)。编译器会拒绝非仿射的扫描时序，也会拒绝那些在不同扫描点上边沿顺序会交换的模板 (此时应拆成多个 chunk，或每个点单独 prepare)。

7.4 端到端的可运行示例

下面扫描 period 0 的持续时间 (slot s0) 和 period 2 上 da_dipole 总线的 DAC 值 (slot s1)，同时给 probe channel 设一个**固定**延时 (500 ns，不是扫描量)：

Python

```
import numpy as np
import Zou_lab_control.neutral_atom as na

state =
↳ na.PulseTableState.load("pulses/camera_imaging_address_switch.json")

# probe 的延时是一个 FIXED 的每通道值 ——不进扫描表
state.delays["probe"] = 500.0 # 固定 500 ns 输出延时

# 绑定两个可扫字段 -> s0, s1 (bind_field 是幂等的, 返回 slot 序号)
s0 = state.bind_field("duration", "0") # period 0 的持续时间
s1 = state.bind_field("dac", "da_dipole@2") # period 2 上 da_dipole 的
↳ DAC 码

# 构造 5 个扫描点的 (N_points x 2) 表: 第一列是 ns, 第二列是带符号 DAC 值 (0 = 0 V)
points = np.linspace(1_000, 5_000, 5)
codes = np.linspace(-512, 511, 5).astype(int)
scan_table = np.column_stack([points, codes])
state.set_scan_table(scan_table) # 也可
↳ state.load_scan_table("scan.npy")

# 编译成扫描 program 并上传
program = state.compile_scan(clock_hz=exp.devices.sequencer.clock_hz)
exp.devices.sequencer.prepare(program)
exp.devices.sequencer.fire()
exp.devices.sequencer.wait_done()
```

编译产物 RuntimeSequenceProgram 在 schema 里携带 slot_count、slot_kinds、tick_slot_coeffs、loop_end_slot_coeffs、scan_points 和 scan_coeff_frac_bits，再加上常规的 ticks/masks/bus_segments。FPGA 一次上传后无缝迭代所有扫描点，每点对外触发一次相机。

7.5 扫描 DAC 值 (硬件无缝)

模拟总线 (DAC) 的值也能逐点无缝扫描, 和数字时序扫描共用同一张扫描点表, 不需要每点重新 prepare。绑定 `kind="dac"`, `scan_table` 的对应列填**原始整数 DAC code** (不是电压、不是 ns):

Python

```
import numpy as np
import Zou_lab_control.neutral_atom as na

state =
↳ na.PulseTableState.load("pulses/camera_imaging_address_switch.json")

# 把 da_dipole 在 period 2 的 DAC 电平绑成一个扫描 slot。
sd = state.bind_field("dac", "da_dipole@2") # target =
↳ "<bus>@<period_index>"

# 11 个点, 带符号 10-bit DAC 值 (-512..+511, 0 = 0 V)。
state.set_scan_table(np.linspace(0, 1000, 11).astype(int).reshape(-1, 1))

program = state.compile_scan(clock_hz=exp.devices.sequencer.clock_hz)
exp.devices.sequencer.prepare(program)
exp.devices.sequencer.fire() # 逐点切 DAC,
↳ 每点触发一次相机
```

底层上, 被扫的 DAC 段在边沿模板里不占数字边沿, 而是变成一个带 `value_select` 的总线段: FPGA 在每个扫描点从对应 slot 读 DAC code 写到总线输出。DAC slot 的列在扫描点表里存原始 code (不做 ns→tick 换算), 其仿射系数恒为 0, 所以它从不进入 `effective_tick` 公式。机制细节见 FPGA 手册“无缝扫描 DAC 值”一节。

组合规则

延时不是扫描量——它是每条 channel 的一个**固定值**, 不进扫描表; 它会作为一条加在输出上的**逐通道延时与任意扫描共存** (duration 扫描、DAC 扫描都行)。可以扫的只有 **DAC 值与持续时间**, 两者**可以同时扫** (不再互斥)。被扫的 DAC 值既可以是 `edge` 的目标电平, 也可以是 `ramp` 的端点 (斜坡在运行时从携带入值斜变到该扫描点的值——两个端点都支持运行时读扫描槽)。延时 (正/负/零) 大小由一个 32 位字段决定 ($\approx \pm 42.9$ s, 毫秒级 emCCD 延时直接可用), 对 TTL 和 DAC **完全一样**——详见下一节。

7.6 固定延时如何与扫描共存 (事件调度的逐信号延时)

channel 延时**不可扫描**: `SCAN_SLOT_KINDS = ("duration", "dac"), state.bind_field("delay", ...)` 会抛 `ValueError("delay is a fixed per-channel value and cannot be scanned")`。延时是每条 channel 的一个**固定值**, 通过 `state.delays[channel] = ...` (配 `state.delay_units`) 设定, 编译时作为常数携带 (`channel_delays`)。

它在硬件上被实现成加在引擎**输出**上的**逐信号事件调度器**: `output_delayed[t] = output_undelayed[t-d]`, `t < d` 之前恒为 0。它有三条关键性质:

- **范围很大 (覆盖到秒级)**。延时 d 可以是正、负、零, 且**与帧长无关** (可大于一帧、可小于一个 tick); 上限是一个 32 位字段 ($\approx \pm 42.9$ s)。真正的资源约束不是延时长, 而是在**途翻转/值变化数**——每个信号的事件 FIFO 深 `EVT_DEPTH` (TTL) / `BUS_EVT_DEPTH` (DAC), 主机对完整日程 (含扫描、bracket、无限循环回绕、跨多帧的长窗口) 精确计数; 超出时在 `validate`/打包阶段 (上

硬件之前) 抛出清楚错误, GUI 也会把单个输入夹回 32 位上限。

- **物理延时、第一帧真实。**这是真实的物理延时, 不是 GUI 预览里那种循环 $%T$ 旋转: fire 之后那条 channel 安静到 $t = d$ 才开始, **第一帧是真实的** (没有“绕回来的尾巴”幻影)。负延时把整个序列重新平移一个全局量 G 。
- **绝不打扰别的 channel、与扫描均匀组合。**延一条 channel 不改变任何其它 channel 的时序, 也不改变周期长度; 延时与 duration 扫描、DAC 扫描、内层 repeat bracket **均匀组合**——“延时后的序列”恒等于“把那条 channel 延时 d 的无延时序列”。

DAC 总线有完全相同的能力: 一个 DAC 值 (固定、被扫、或斜坡 ramp) 都能用同样的方式延时 (同一个 32 位范围), $t < d$ 之前保持安全值, 延时**超过一帧**也支持; 一条 DAC 总线的 10 个 bit **共享一个**延时。机制细节 (事件调度器、启动门、容量契约) 见 FPGA 手册“逐信号输出延时 (事件调度器)”一节。

7.7 扫描 + 采集：一次完整循环

硬件无缝扫描时, FPGA 在一次 `fire()` 里跑完所有扫描点, 每点对外触发一次相机。所以典型用法是: arm 相机收 `N_points` 帧, prepare/fire 扫描 program, 再按点 reshape 结果。

Python

```
import numpy as np
import Zou_lab_control.neutral_atom as na

exp = na.connect("remote_template", sequencer={"host": "192.168.0.20",
↪ "port": 18861}, open_devices=True)
state =
↪ na.PulseTableState.load("pulses/camera_imaging_address_switch.json")

s0 = state.bind_field("duration", "1")           # 扫成像段时长
points = np.linspace(1_000, 5_000, 21)          # 21 个点, ns
state.set_scan_table(points.reshape(-1, 1))

program = state.compile_scan(clock_hz=exp.devices.sequencer.clock_hz)
exp.camera.arm(frames=len(points))                # 先 arm 刚好这么多帧
exp.devices.sequencer.prepare(program)
exp.devices.sequencer.fire()                      # 无缝跑完 21 个点
images = exp.camera.read()                        # (21, H, W)
```

要点: 用**有限**扫描 program (不要 `repeat_forever`), 让相机的帧数边界和扫描点数对齐; 不要让 qCMOS 等一个没有帧数边界的自由循环脉冲。

7.8 解绑与单点求值

Python

```
state.unbind_slot(s1)                             # 删除 slot; 后面的 slot 自动顺移 (s2 ->
↪ s1, ...)

# 不做硬件扫描、只想用一组常数跑一次时:
one_shot = state.with_slots_resolved({"s0": 2_000}) # 把 s0 替换成 2000 ns
↪ 的常数副本
```

8

分层与状态归属（深入）

本章把整个 `Zou_lab_control.neutral_atom` 控制框架拆开，逐层讲清楚**每一层负责什么、每一份状态归谁所有、谁有权改它、生命周期是怎样的**。这套系统看上去很大，但它的设计哲学其实只有一句话：**时间真相、硬件动作、纯算法、实验编排各居其位，互不越界**。读完本章，一个第一次接触这个仓库的人应该能够：理解每一层的职责边界、用 `na.connect` 拿到一个会话对象、知道这个对象里挂着哪些状态、在三种后端（虚拟/本地/远程）之间切换、并亲手跑通一次最小实验。

8.1 为什么要分层：一句话的设计哲学

控制一个中性原子实验，本质上要同时管好四类完全不同的东西：

- **时间**：什么通道在什么时刻翻转、脉冲多长、扫描参数怎么平移每条边沿。这是确定性的、可编译的“真相”。
- **硬件**：相机怎么曝光、FPGA 怎么收脉冲表、JTAG-to-AXI 怎么写 BRAM 和控制寄存器。这是有副作用、会失败、需要连接的“动作”。
- **算法**：从一张原始图里找出原子、做校准拟合、判定每个格点亮不亮。这是纯函数式的、可离线复现的“计算”。
- **编排**：把上面三者串成“准备序列 → 开火 → 读出 → 分析 → 存历史”的实验循环。这是“流程”。

如果把这四类混在一起（比如让相机驱动顺手画出格点圆圈、让设备类里塞拟合算法），系统很快就会变得无法测试、无法离线复现、无法换后端。因此本框架把它们**物理隔离**成不同的子包，并用几条铁律（见本章最后一节“不变量”）锁死边界。

8.2 六个子包：职责与状态归属

`Zou_lab_control/neutral_atom/` 下分成六个目录。下面逐个说明它“做什么”和“拥有什么状态”。

8.2.1 devices/ ——硬件适配器与契约

做什么 把每一台真实仪器包装成一个统一接口的 Python 对象，**只做硬件动作**：连接、配置、触发、读回字节。它不懂算法，也不懂实验流程。

关键文件 `axi_session.py` (VivadoAxiStreamerSession: JTAG-to-AXI 打包上传 + CTRL mailbox), `fpga_pulse_streamer.py` (XDC 推断 + `validate_pulse_streamer_program`), `sequencer.py` (RuntimeSequenceProgram, SequencerService, RemoteSequencer), `sequencer_server.py` (run_server, CommandSequencerBackend), `qcmos.py` (qCMOS 相机适配器), `virtual.py` (离线假设备), `registry.py` (设备注册/查找), `base.py` (设备基类与契约)。

拥有的状态 只有**与硬件连接相关**的状态：打开的句柄、hw_axi 会话、当前已写入硬件的脉冲程序、相机的曝光参数。它**不**持有图像分析结果，也**不**持有校准。

这里有一条贯穿全书的契约：**TrapArrayDevice 暴露 n_sites (有多少个格点)，但绝不暴露每个格点在相机上的像素中心**。像素中心是几何/光学事实，属于校准 (TrapCalibration)，不属于设备。设备只知道“我这套光阱有 N 个位点”，至于这 N 个点落在相机的哪些像素上，要去问校准。

8.2.2 timing/ ——时间真相

做什么 定义脉冲序列的“真相”，并把它编译成 FPGA 能吃的边沿表。这是整个系统里唯一允许定义“时间是什么”的地方。

关键文件 `sequence.py` 里的 `PulseSequence` (程序化构造序列的核心数据结构); `pulse_table.py` 里的 `PulseTableState` (GUI 用的编译模型，承载周期/通道延迟/DAC 总线段/扫描槽 `scan_slots` 与扫描表 `scan_table`); `verilog.py` (生成微型边沿表)。

拥有的状态 当前序列的全部时间定义：周期、每通道延迟、总线段、扫描槽绑定与扫描点表。它**不**碰硬件，编译产物 (`RuntimeSequenceProgram`) 交给 devices 层去上传。

时间层是**纯数据 + 纯编译**的。你可以在完全没有硬件、没有 Vivado 的机器上构造一个 `PulseSequence`、调用编译、检查产物，这正是单元测试和离线开发的基础。

8.2.3 operations/ ——纯算法 (离线)

做什么 收纳所有**纯函数式**的图像处理、校准拟合、原子检测算法。给定输入数组，返回结果，**没有任何副作用**，不连接任何硬件。

拥有的状态 几乎没有。它是算法的家，不是数据的家。输入从外面传进来，结果返回出去。

把算法关在这里有两个巨大好处：一是**可离线复现**——拿一张存档的原始图，离线就能重跑检测，得到逐位元一致的结果；二是**可单测**——算法测试不需要相机、不需要网络。

8.2.4 subsystems/ ——实验级编排

做什么 把 devices 的硬件动作和 operations 的算法**编排**成实验级能力。这一层是会话对象上 `exp.readout`、`exp.timing` 背后的实现。

关键成员 `ReadoutSubsystem` (读出子系统：触发相机、拿原始图、调 operations 做检测、产出每格点占据判定), `TimingSubsystem` (时序子系统：在会话级别管理当前序列的准备与开火)。

拥有的状态 编排所需的中间状态与对底层设备/算法的引用，而非原始硬件句柄本身。

8.2.5 views/ ——给前端的绘图适配器

做什么 把内部数据结构转换成前端（GUI/笔记本）能直接画的形式。它是“展示适配器”，不产生新数据。

拥有的状态 无持久状态；它是从模型到可视化的纯转换层。

8.2.6 core/ ——校准与结果模型

做什么 定义跨实验复用的核心模型与分析入口。

关键文件 `calibration.py` 里的 `TrapCalibration`（把格点编号映射到相机像素几何，即前面说的“像素中心”的真正归属地），`analysis.py`（分析流程），`results.py`（结果数据结构）。

拥有的状态 校准对象本身是一份可被会话持有、可保存/加载的状态。它是连接“设备说有 N 个格点”和“图像上这 N 个点在哪”的桥梁。

一句话记住分层

timing 说“时间是什么”，**devices** 说“怎么对硬件动手”，**operations** 说“怎么从数据算出答案”，**subsystems** 把它们编排成实验能力，**core** 提供校准与结果模型，**views** 负责画出来。没有任何一层越界去做别人的事。

8.3 na.connect 与会话对象

整个框架对用户暴露的入口非常窄：你调用 `na.connect(config)`，拿到一个 `NeutralAtomSession`。这个会话对象就是你与整个实验交互的唯一句柄——所有状态都挂在它身上，所有操作都从它出发。

Python

```
import Zou_lab_control.neutral_atom as na

# config 选择后端与硬件参数；下一节详述三种后端
session = na.connect(config)
```

8.3.1 会话拥有哪些状态

`NeutralAtomSession` 是这套系统的**状态归属中心**。它持有以下几份状态，每一份都有明确的所有者语义：

devices 一个 `DeviceSet`：本次会话连接的全部设备（相机、序列器等）的集合。设备的硬件句柄归它管。

sequence 当前的 `PulseSequence`：本会话此刻的时间真相。你修改序列、编译序列，改的都是这一份。

_calibration 一个 `TrapCalibration` 或 `None`：当前校准。下划线前缀表示它是受控状态——通过会话的接口设置/使用，而不是随便外部赋值。没校准时为 `None`。

history 一个列表：本会话历次实验产出的结果，按时间追加。它是你这次实验跑了什么的“账本”。

readout 一个 `ReadoutSubsystem`：读出能力的实现，即 `exp.readout` 背后的对象。

timing 一个 `TimingSubsystem`：时序能力的实现，即 `exp.timing` 背后的对象。

请注意所有权的清晰划分：**时间真相归 sequence**，**硬件归 devices**，**校准归 _calibration**，**历史归 history**。读出子系统在工作时会去问设备要原始图、问校准要几何、调 operations 做检测，但它**不拥有**这些状态，只是编排它们。

8.3.2 一个最小会话的全貌

Python

```
import Zou_lab_control.neutral_atom as na

session = na.connect(config)

# 会话身上挂着的状态:
session.devices          # DeviceSet: 本次连接的硬件
session.sequence         # 当前 PulseSequence (时间真相)
session.history           # list: 历次结果的账本

# 会话暴露的实验能力 (编排层):
session.readout           # ReadoutSubsystem
session.timing            # TimingSubsystem
```

8.4 三种后端，同一张会话脸

本框架最重要的工程价值之一：**三种后端共享完全相同的会话表面**。也就是说，无论你连的是离线虚拟设备、本地真实硬件、还是远程 FPGA，你拿到的 `NeutralAtomSession` 的接口一模一样。换后端只换 `config`，不改一行实验脚本。

虚拟后端（离线） 全部用 `virtual.py` 里的假设备。没有相机、没有 Vivado、没有网络，照样能 `na.connect`、构造序列、跑读出（用合成图）、走完整条流程。这是开发、教学、回归测试的主力。

真实 qCMOS + 本地 RuntimeSequencer 真实 `qcmos` 相机 + 本机的 `RuntimeSequencer`。脉冲程序在本地编译并驱动硬件，适合相机和时序硬件都接在同一台机器上的情形。

真实 qCMOS + 远程 FPGA (RemoteSequencer / RPyC) 真实相机在本地，脉冲流送给**远程** FPGA：本地的 `RemoteSequencer` 通过 RPyC 调到 `sequencer_server.py` 跑的服务 (`run_server`, jtag-axi 后端)，由它经 `VivadoAxiStreamerSession` 用 JTAG-to-AXI 操作 FPGA。这是真机实验的典型部署。

换后端只换 config

因为三种后端共享同一张会话脸，你完全可以**先在虚拟后端把整个实验脚本写好测好**，再把 `config` 切到真机后端原样跑。这正是“同一会话表面”设计带来的最大红利。

Python

```
# 同一段实验脚本，三种后端通吃 —— 只有 connect 的 config 不同。
import Zou_lab_control.neutral_atom as na

session = na.connect(config)          # config 决定虚拟 / 本地 / 远程

session.timing.prepare(session.sequence) # 把时间真相写进（虚拟或真实的）序列器
result = session.readout.run()          # 触发开火 + 读出 + 检测
session.history.append(result)           # 记入账本
```

8.5 走通一次真实实验：完整生命周期

下面把一次实验从头到尾的生命周期讲清楚。每一步都标注“谁拥有这一步动的状态”，这样你能始终知道自己在和哪一层打交道。

1. **连接**: `na.connect(config)` 创建会话, 连上设备 (`session.devices`), 初始化空历史、空校准、读出与时序子系统。
2. **定义时间**: 构造或修改 `session.sequence` (一个 `PulseSequence`)。这是 timing 层的事, 纯数据、无副作用。
3. **准备序列**: `session.timing.prepare(...)` 把时间真相编译成 `RuntimeSequenceProgram` 并写入序列器硬件 (本地或远程)。从这一步起才真正“碰硬件”。
4. **校准 (按需)**: 若还没有校准, 先做一次得到 `TrapCalibration`, 让会话持有它。读出要靠它把格点编号映射到相机像素几何。
5. **读出**: `session.readout` 触发开火、采集**原始图**、调用 operations 的检测算法、结合校准产出每个格点的占据判定。
6. **记账**: 把本次结果追加进 `session.history`。
7. **分析**: 用 `core/analysis.py`、`operations/` 对历史结果做离线分析——这些都是纯算法, 可随时重跑。

8.5.1 一段可运行的最小实验

Python

```
import Zou_lab_control.neutral_atom as na

# 1) 连接 (虚拟后端先跑通, 再换真机 config)
session = na.connect(config)

# 2) 定义时间真相: 当前序列就是 session.sequence
seq = session.sequence
# .....在此用 PulseSequence 的接口编辑周期 / 通道 / 扫描槽.....

# 3) 准备: 把时间真相编译并写入序列器
session.timing.prepare(seq)

# 4) 读出: 开火 + 采原始图 + 检测 + 结合校准判定占据
result = session.readout.run()

# 5) 记账
session.history.append(result)

# 6) 离线分析历史 (纯算法, 无副作用, 可复现)
# 用 operations/ 与 core/analysis.py 处理 session.history
```

先在虚拟后端跑这一段

上面这段脚本在虚拟后端就能完整跑完: 序列器是假的、相机是假的、图是合成的, 但流程、接口、历史账本都和真机一致。确认逻辑没问题后, 把 `config` 换成真机后端再跑一次即可。

8.6 铁律：把边界焊死的不变量

分层只有靠不变量来强制执行才有意义。本框架有几条**绝不能破**的规则。它们看起来是细节, 实则是整套架构能换后端、能离线复现、能单测的根本原因。

capture 只给原始图 `capture()` 返回的**只有原始图像**，不带任何叠加。格点圆圈、占据标记这些**属于读出 (readout)，不属于相机**。相机驱动的职责到“把像素读回来”为止，再往上一步都不归它。

设备只做硬件动作 `devices` 层只负责硬件动作，**所有算法都住在 `operations/` 与 `core/`**。你不在相机类里看到拟合，也不会在序列器类里看到检测。

TrapArrayDevice 不知道像素中心 `TrapArrayDevice` 暴露 `n_sites`，但**不暴露**相机像素中心。像素中心来自 `TrapCalibration` (在 `core/calibration.py`)。设备知道“有几个格点”，校准知道“它们在像素上的哪里”——两份知识分属两层。

为什么这几条这么重要

这三条不变量合起来保证了：相机可以被换成虚拟设备而读出逻辑不变（因为读出不依赖相机去画圈）；算法可以离线对存档图重跑（因为算法不在设备里）；几何校准可以独立保存、加载、复用（因为它不被塞进设备状态）。**每当你想在某一层“顺手多做一点别人的事”时，回到这三条铁律——答案几乎总是“不要”。**

8.7 把分层装进脑子里：一张归属速查表

最后用一张速查表收尾，帮你在写代码时随时确认“这件事归谁、这份状态在哪”。

我要改脉冲时长 / 通道延迟 / 扫描 改 `session.sequence(timing层, sequence.py / pulse_table.py)`。

我要把序列送上硬件 用 `session.timing.prepare(...)` (`subsystems` 编排 → `devices` 上传，本地 `RuntimeSequencer` 或远程 `RemoteSequencer`)。

我要拍一张原始图 找相机 `capture()` (`devices/qcmos.py`) ——只会拿到原始像素，没有圈。

我要知道某格点在图上哪个像素 问 `TrapCalibration` (`core/calibration.py`)，不要去问 `TrapArrayDevice`。

我要从图里找原子 / 拟合 / 判定占据 用 `operations/` 的纯算法；在实验流程里由 `session.readout` 编排调用。

我跑过的实验结果在哪 `session.history`。

我想离线复现一次分析 拿存档原始图喂给 `operations/` 与 `core/analysis.py`，不需要任何硬件。

我想换后端 只改 `na.connect` 的 `config`，脚本一字不改。

记住会话对象是一切的中心：**`na.connect(config)` 给你一个 `NeutralAtomSession`，它拥有 `devices / sequence / _calibration / history`，并通过 `readout` 与 `timing` 两个子系统把硬件、时间、算法编排成你能跑的实验**。守住分层与不变量，这套系统就能在虚拟、本地、远程三种后端上以同一张脸为你工作。

9

PulseSequence 与 PulseTableState (深入)

整个时序系统里有两个名字很像、但绝对不能混淆的对象：**PulseSequence** 和 **PulseTableState**。它们分工不同、拥有的状态不同、生命周期也不同。读完这一章，一个新人应该能说清楚：谁是“物理真相”，谁是“人类编辑模型”，一行边沿表到底是什么意思，以及如何从一个 GUI 表格一路编译到可以上传给 FPGA 的边沿表，并据此跑一次真实实验。

9.1 为什么是两个模型

实验控制有两套截然不同的需求，硬塞进一个类里会两头不讨好，所以代码刻意把它们拆成两层。

底层真相 (PulseSequence) 定义在 `Zou\_lab\_control/neutral\_atom/timing/sequence.py`。

它是**时间正确**的唯一真相：它知道每一个通道在任意时刻的电平。它不关心“周期”“人类标签”“GUI 可见性”这些概念，只关心一组脉冲（某通道在 start 秒开始、持续 duration 秒为高），以及每通道的整体延迟。它可以把自己编译成一张**边沿表**（绝对 tick + 完整输出掩码）。Notebook、PyQt 前端、远程 FPGA 三方共享的就是这一层。

编辑模型 (PulseTableState) 定义在 `Zou\_lab\_control/neutral\_atom/timing/pulse\_table.py`。

它是 GUI 的**编译模型**，保存的是给人看的概念：一行一行的**周期 (PERIOD)**，每个周期有一个时长加一个数字状态向量；还有通道名/标签、可见性、每通道延迟、模拟总线方案（analog-bus plan）、以及扫描槽（scan slots）。它**不拥有任何硬件**，它只负责把这些人类概念 `compile()` 成一个 PulseSequence / 运行时程序。

一句话区分

PulseSequence 回答“在第几纳秒，哪些通道是高”；PulseTableState 回答“这张表格里，第几个周期持续多久、哪些灯亮”。前者是物理，后者是界面。PulseTableState 最终**下沉**成 PulseSequence，再下沉成边沿表。

这样拆分的好处：

- GUI 可以自由地引入“周期”“延迟框”“可见通道”这些纯展示概念，而不会污染时间真相。
- Notebook 用户可以完全绕过表格，直接用 `PulseSequence.pulse(...)` 手写脉冲，得到的边沿表和 GUI 编出来的是同一种东西、走同一个上传管线。

- 只有 PulseSequence 这一层需要为“时间是否落在时钟网格上”“掩码怎么打包”负责；PulseTableState 只要保证自己能干净地下沉即可。

9.2 一行边沿表是什么意思

边沿表 (edge table) 是这套系统真正交给硬件的东西。理解它，整条链路就通了。

一行边沿 = “在这个绝对 FPGA tick，把所有输出一次性切换到这个掩码”。

- **绝对 tick**: 从序列起点 $t=0$ 算起的时钟周期计数。50 MHz 时钟下，一个 tick = 20 ns; tick=100000 就是 $t = 100000 \times 20 \text{ ns} = 2 \text{ ms}$ 。
- **掩码 (mask)**: 一个整数，按位表示这一刻**所有**通道的电平。**bit 0 = channels[0]**, bit 1 = channels[1], 依此类推。掩码是“全量快照”，不是“增量”——每一行都重新写满所有通道的当前电平。
- **合并**: 如果多个通道在**同一时刻**一起变化，它们**合并成同一行**的同一个掩码。硬件每个 tick 只换一次输出，所以同一瞬间的所有变化天然合到一行。

举一个相机成像预设 在 50 MHz 下编译出来的真实例子。假设通道列表里 ch00=cooling (冷却)、ch03=probe (探测)、ch09=trap (阱)、ch11=emCCD (相机触发位)。编译结果大致是：

text

```
ticks = [0,      100000, 105000, ...]
masks = [513,    512,    2568, ...]
```

逐行解读：

- **tick=0, mask=513**。513 的二进制是 1000000001，即 bit0 + bit9 = ch00 + ch09 = **cooling + trap** 同时为高。序列一开始就把这两路打开。
- **tick=100000, mask=512**。512 = bit9 = 只剩 ch09=trap。也就是说在 tick 100000 (50 MHz 下 = 2 ms) 那一刻，cooling 关掉了，trap 仍开。注意 cooling 的“关”是**隐式**的——掩码里那一位变成 0 就是关。
- **tick=105000, mask=2568**。2568 = bit3 + bit9 + bit11 = ch03 + ch09 + ch11 = **probe + trap + emCCD**。这一刻探测光打开、阱保持、相机触发位拉高。emCCD 就是相机触发位——它出现在掩码里，相机就在这一 tick 被触发曝光。

为什么掩码是全量而不是增量

全量掩码让硬件极其简单：每到一个边沿 tick，直接把这个整数怼到输出寄存器，不需要“记住上一刻是什么再做异或”。代价是每行要写满所有位，但通道数固定 (62 条)，整数装得下，存储也省。

9.3 从 PulseSequence 到边沿表：compile_runtime_program

把一个 PulseSequence 变成可上传程序的函数是 `compile_runtime_program(sequence, *, channels, clock_hz)`，定义在 `Zou\_lab\_control/neutral\_atom/devices/sequencer.py`。它返回一个 `RuntimeSequenceProgram`。

它内部做的事，按顺序：

1. 规整通道列表，校验 clock_hz 为正。
2. 取 `sequence.without_repeat()` 得到“一份未展开的基础序列”，再调用 `base_sequence.edges(clock_hz=..., channels=...)` 得到 (ticks, masks, channels)——这就是上面讲的边沿表。

3. 计算 loop_end_tick: 如果序列设了重复周期 repeat_period, 就用它换算成 tick; 否则用最后一条边沿的 tick。这决定了硬件循环回绕的位置。
4. 用 `_ensure_final_off_edge(...)` 在循环末尾补一条“全关”边沿, 保证回绕前输出干净。
5. 把 sequence、clock_hz、channels、ticks、masks 等打包, 算一个 sha256 截断作为 `sequence_id` (用于去重/缓存)。
6. 组装并返回 RuntimeSequenceProgram, 里面带上 ticks/masks、duration、repeat_forever、loop_start_index、loop_end_tick、loop_count 等元数据。序列器是**纯脉冲流送器**: 它只编译并流送边沿表, 不知道哪条通道是相机触发, 也不在程序里数触发次数。**要数一段序列里相机被触发几次** (每次触发出一帧), 用相机层的 `count_trigger_pulses(sequence, trigger_channels=...)` (定义在 `Zou_lab_control_neutral_atom_devices_camera_trigger.py`), 并传入**相机自己的** `capture_trigger_channels`——这条 TTL 线接哪是相机的属性。

关键点: 边沿表本身**不把重复展开**。一个会重复 1000 次的成像循环, 边沿表只存一份, 配上 loop_end_tick 和 loop_count, 让硬件自己回绕。这就是为什么内部先 `without_repeat()` 拿基础边沿, 再单独带上循环元数据。

一个纯 Notebook、不碰 GUI 的最小例子:

Python

```
from Zou_lab_control.neutral_atom.timing.sequence import PulseSequence
from Zou_lab_control.neutral_atom.devices.sequencer import
↳ compile_runtime_program

# 手写脉冲: cooling 从 0 开 2ms; trap 全程开 2.1ms; probe+emCCD 在 2.1ms 触发
↳ 0.1ms
seq = (
    PulseSequence(name="imaging")
    .pulse("ch00", start=0.0, duration=2.0e-3) # cooling
    .pulse("ch09", start=0.0, duration=2.1e-3) # trap
    .pulse("ch03", start=2.1e-3, duration=0.1e-3) # probe
    .pulse("ch11", start=2.1e-3, duration=0.1e-3) # emCCD 相机触发
)

prog = compile_runtime_program(
    seq,
    channels=[f"ch{n:02d}" for n in range(62)], # bit0=ch00, bit1=ch01,
    ↳ ...
    clock_hz=50e6, # 20ns/tick
)

print(prog.ticks) # 绝对 tick 列表
print(prog.masks) # 与 ticks 对齐的全量掩码列表

# 数相机触发个数 (每次触发出一帧) 是相机层的事: 用 count_trigger_pulses,
# 传相机自己的 capture_trigger_channels (这里 emCCD 接在 ch11)。
from Zou_lab_control.neutral_atom.devices.camera_trigger import
↳ count_trigger_pulses
print(count_trigger_pulses(seq, trigger_channels=["ch11"])) # = 1 (emCCD
↳ 触发了一次)
```

这里 `channels` 的**顺序就是位序**: `channels[0]`→bit0。如果你把通道列表换个顺序, 同样的脉冲会

编出不同的掩码整数——但物理上等价。

9.4 PulseTableState 的状态与 API

PulseTableState 是 GUI 那张表的内存模型。它拥有的状态（谁拥有什么）：

periods 一串周期，每个周期 = 一个时长 + 一个数字状态向量（哪些通道在这个周期为高）。这是表格的每一行。

channels / channel_labels / visible_channels 通道名、给人看的标签、以及哪些通道在 GUI 里可见（纯展示，不影响编译出的物理）。

delays / delay_units 每通道的整体延迟，以及它的单位。延迟会在下沉成 PulseSequence 时整体平移该通道。

analog_buses / analog_bus_modes 模拟总线(DAC 总线)的成员通道, 以及每个周期该总线是“edge (写一个值)”还是“hold (保持)”、写什么值。

scan_slots / scan_table 扫描槽(有序, 每个绑定到一个 **duration** 或 **dac** 字段——**SCAN_SLOT_KINDS** = (“duration”, “dac”), 延时不可绑) 和扫描表 ($N_{\text{points}} \times N_{\text{slots}}$)。表达式里用 $s0..s\{N-1\}$ 引用这些槽。

核心方法（都在 `Zou\_lab\_control/neutral\_atom/timing/pulse\_table.py`）：

bind_field(kind, target, *, label, unit, nominal) 把一个字段绑定到**新的扫描槽**，并把该字段原地改写成 $s\{i\}$ (i 是新槽号)，返回槽号。kind $\in \{\text{duration}, \text{dac}\}$ ：duration 的 target 是周期下标，dac 的 target 指向某条总线的某个周期。**延时不能绑定**——`bind_field("delay", ...)` 抛 `ValueError("delay is a fixed per-channel value and cannot be scanned")`；延时通过 **delays** 设成固定值。**幂等**：重复绑定同一个已绑定字段，直接返回它已有的槽号。绑定时会给 scan_table 每行追加该字段的 nominal 标称值。

unbind_slot(index, *, restore=None) 删除第 index 个扫描槽；后面的槽**整体降号** ($s2 \rightarrow s1$...)，并重新把每个剩余槽的变量写回对应字段。restore 给被解绑字段一个恢复值。

slot_index_for(kind, target) 查某个字段当前绑到了哪个槽，没绑返回 None。bind_field 的幂等就是靠它实现的。

set_scan_table(rows) / load_scan_table(path) 设置/从文件载入扫描表，会按当前槽数 normalize。表是 N_{points} 行 $\times N_{\text{slots}}$ 列。

with_slots_resolved(slots) 返回一个**非扫描**的副本，把每个槽替换成常量值（时间槽吃 ns、dac 槽吃整数 DAC 码）。用于“每发设一个值”的简洁 Notebook 单点扫描。

analog_bus_plan(name) / set_analog_bus_mode(...) / **set_bus_value(period, bus, value)** 读/写某条模拟总线的逐周期方案 (edge/hold + 值)。

to_sequence(...) 把这张表下沉成一个 PulseSequence（遍历周期，把每段连续为高的区间变成 `seq.pulse(...)`，再对每通道套上延迟）。这是 PulseTableState $\rightarrow$ PulseSequence 的桥。

compile(*, clock_hz, slots, repeat_forever) 把表（解析掉扫描槽后的单点）编译成一个静态运行时程序。内部转去 `compile_pulse_table_runtime_program`。

compile_scan(*, clock_hz, scan_table, repeat_forever) 把表连同扫描表一起编译成一个**带扫描**的运行时程序(硬件逐点扫)。内部转去 `compile_pulse_table_scan_runtime_program`。

槽号是位置而非名字

扫描槽是**有序**的，表达式只能写 s0、s1.....不能起别名。所以 `unbind_slot` 会触发降号重排——删了 s1，原来的 s2 就变 s0 之后的 s1。这一点在手写表达式或读 `scan_table` 列时务必记住：列顺序严格对应 `scan_slots` 的顺序。

9.5 一个完整的 PulseTableState 编译实例

下面从零搭一张成像表，绑一个扫描槽，分别走**单点编译**和**扫描编译**两条路，最后说明怎么据此跑实验。

Python

```
from Zou_lab_control.neutral_atom.timing.pulse_table import
↳ PulseTableState

# 1) 建表: 62 个通道, 3 个周期
state = PulseTableState(channels=[f"ch{n:02d}" for n in range(62)])

# 周期 0: cooling+trap 亮 2ms; 周期 1: 仅 trap 5us; 周期 2: probe+trap+emCCD 亮
↳ 0.1ms
# (states 是数字状态向量: 哪些通道在该周期为高。具体填法见 GUI / period API)
# 这里假设已经通过界面/构造把三行周期填好, 时长分别约 2ms / 5us / 0.1ms。

# 2) 把" 周期 0 的时长" 绑成一个扫描槽 ——我们要扫冷却时间
slot = state.bind_field("duration", "0", unit="ns", nominal=2_000_000.0)
print("cooling 时长绑到槽:", slot)    # -> 0, 字段现在是 's0'

# 3) 给扫描表: 扫 5 个冷却时长 (单位 ns), 一列对应 s0
state.set_scan_table([[1_000_000.0],
                      [1_500_000.0],
                      [2_000_000.0],
                      [2_500_000.0],
                      [3_000_000.0]])

# 4a) 单点编译: 把 s0 解析成一个常量, 得到一份静态程序
prog_single = state.compile(
    clock_hz=50e6,
    slots={"s0": 2_000_000.0},    # 这一发用 2ms 冷却
)
print(prog_single.ticks[:4], prog_single.masks[:4])

# 4b) 扫描编译: 把整张扫描表交给硬件逐点扫
prog_scan = state.compile_scan(clock_hz=50e6)
print(" 扫描点数:", len(prog_scan.scan_points))    # -> 5
```

两条路的区别:

- `compile(...)` 走 `slots` 把每个 `s{i}` 钉成常量, 得到**一份**静态边沿表——适合“我就跑这一个参数”。
- `compile_scan(...)` 把 `scan_table` 一起带上, 编出的 `RuntimeSequenceProgram` 里有 `scan_points` 和 per-edge 的槽系数, 硬件能**无缝**地一点一点扫过去, 不必每点重新上传。

9.5.1 编出来之后怎么跑实验

无论走哪条路,最终拿到的都是一个 RuntimeSequenceProgram,它和 `compile_runtime_program` 直接编 PulseSequence 得到的是**同一种东西**,因此走同一个上传/运行管线(详见时序与设备相关章节里的 PulseController / 远程 sequencer)。一次最小实验循环大致是:

1. 在 PulseTableState 里搭好周期、通道、延迟、模拟总线方案;用 `bind_field` 绑好你要扫的字段;用 `set_scan_table` 给扫描数组。
2. 调 `state.compile_scan(clock_hz=50e6)` 得到运行时程序(或单点用 `compile`)。
3. 把程序交给设备层(prepare: 拉住复位、按行写入、释放; fire: 发起脉冲; wait_done: 轮询 done)。
4. 每个扫描点对应一发,相机由掩码里的 emCCD 触发位(如上例 ch11)在对应 tick 自动触发曝光;读出与扫描点一一对应。
5. 跑完回到 safe_state。

新人最容易踩的三个坑

1. **别把两个模型搞反**: 你在 GUI 里编辑的是 PulseTableState; 真正上硬件的是它 compile 出来的边沿表 (PulseSequence → RuntimeSequenceProgram)。改了表不 compile, 硬件什么都看不到。
2. **掩码是位序, 不是名字**: 513 之所以是 cooling+trap, 完全取决于 channels 列表的顺序 (bit0=channels[0])。换了通道顺序, 同一物理动作的掩码整数会变。
3. **时间必须落在时钟网格上**: 50 MHz 下一切时长/延迟最终都被换算成整数 tick (20 ns 一格)。不在网格上的时间会在校验阶段报错, 而不是被悄悄取整。

10

Sequencer 生命周期与远程会话（深入）

本章把“在笔记本里调一行 `pulse.on_pulse()`，原子真的被激光照到”这条完整链路拆开，一层一层讲清楚：谁持有什么状态、每个动作在哪台机器上做什么、纳秒级时序到底由谁掌管。读完本章，一个新人应当能独立启动远程服务、连上它、跑出一次有限次相机采集，并理解为什么这套设计在 Windows + Vivado + JTAG 这种“软件层处处有抖动”的环境里仍然能给出确定的脉冲。

10.1 四个动作与两台计算机

整个控制系统只暴露四个真正的硬件动作：**prepare**（准备）、**fire**（发射）、**wait_done**（等待完成）、**safe_state**（回到安全空闲）。它们沿着一条固定的链路传递：

text

```
控制计算机（笔记本/notebook）
  RemoteSequencer      <- 客户端代理，持有 host/port/channels/clock
    | RPyC (TCP, 可选 SSL)
    v
FPGA/Vivado 计算机
  SequencerService      <- 有状态服务，持有 prepared_program + state
    | callback
    v
  VivadoAxiStreamerSession <- 常驻 Vivado Tcl/hw_axi 进程，持有已上传程序
    | JTAG-to-AXI (axi_bram_ctrl + CTRL mailbox)
    v
  zlc_pulse_streamer_top (FPGA) <- 真正掌管纳秒时序的硬件
```

控制计算机 跑实验脚本/笔记本。它只有“意图”：一份脉冲表(`PulseSequence` 或 `PulseTableState`)和扫描表。它不直接碰硬件。

FPGA/Vivado 计算机 接着 FPGA 的那台机器。它跑 RPyC 服务，背后挂一个**常驻**的 Vivado `Tcl/hw_axi` 进程，通过 JTAG-to-AXI 把 BRAM image 写进 FPGA，并读写 CTRL 寄存器 (COMMAND/STATUS mailbox)。

为什么要分两台机器？ 因为 Vivado 启动慢、打开硬件目标 (JTAG 链) 慢、加载探针文件慢。如果每次“On Pulse”都重新启动一遍，每发一枪都要等几十秒。把这些慢操作集中到**服务器启动时**做一次，之后

每次 prepare/fire 只是往一个已经活着的 Tcl 进程里喂几行脚本，毫秒级返回。

10.2 状态归属：谁持有什么

理解这套系统的关键，是搞清楚**每一层各自缓存了什么状态**，以及这些状态在什么时候失效。

RemoteSequencer (客户端) 持有连接参数 (host、port、channels、clock_hz、trigger_channels) 和 `self._conn` (RPyC 连接对象)、`self._last_program` (上一次 prepare 返回的程序快照)。`open()` 在首次使用时建立连接，并用服务器的 `snapshot()` 覆盖本地 channels/clock，保证两端通道顺序与时钟一致。

SequencerService (服务端，进程内状态机) 持有 `self.prepared_program` (当前已准备好的 RuntimeSequenceProgram) 和 `self.state(idle/prepared/running/done/timeout/safe/aborted)`。它用一把 `threading.RLock` 串行化所有动作。`fire()` 前会 `_require_prepared()`，没准备过就直接报错——这是防止“空枪”的护栏。

VivadoAxiStreamerSession (硬件传输层) 持有 `self._process` (常驻 Vivado 子进程)、读取线程、流式回填后台线程，以及当前已载入 FPGA 的程序。它负责把程序打包成 BRAM image、经 JTAG-to-AXI 上传，并驱动 CTRL mailbox。

FPGA (zlc_pulse_streamer_top) 持有边沿表 (abs ticks + 62 位掩码 + slot 系数，三块并行 BRAM)、repeat 元数据、2-bank 扫描点窗口、bus 段。一旦收到 FIRE 上升沿，**枪内的纳秒时序就完全归 FPGA 时钟所有**。

一条铁律：纳秒时序的所有权会“交接”

prepare 阶段，时序由软件主导 (Tcl 把 BRAM image 写进去，多慢都无所谓，因为还没发 FIRE、输出处于安全态)。fire 给出 FIRE 上升沿那一刻，所有权**交接**给 FPGA 时钟。此后 Python 的 GIL、RPyC 往返、Windows 调度抖动、Vivado 的延迟、JTAG 的延迟，**都不再影响一枪之内的边沿时刻**。这就是为什么“在 Python 里 sleep 然后翻通道”绝对不可接受，而“把整张边沿表交给 FPGA 自己跑”可以做到纳秒确定。

10.3 prepare：准备但不发射

`prepare(program)` 做四件事，顺序严格：

1. **先发 SAFE**。让 FPGA 回到安全态 (输出钳在安全电平)，此时往 BRAM 写程序映像不会产生任何脉冲。
2. **校验后打包上传**。`validate_pulse_streamer_program` 先检查边沿数、扫描点数、tick 位宽、通道数不越界，并校验全局有效 tick 单调 (扫描不会把合并后的边沿顺序打乱)；通过后用 `host.image.pack_program` 把整张程序打包成 BRAM image (CTRL 标量 + 三块并行边沿表 + 2-bank 扫描窗口 + bus 段)，经 JTAG-to-AXI 一次写进 `axi_bram_ctrl`。越界或不单调会在写任何 BRAM 之前就报错。
3. **arm 扫描 bank**。对流式扫描，把前两个 chunk 装进 ping-pong 的两个 bank，并通过 `BANK_READY/BANK*_CHUNK` 告知引擎哪个 bank 装的是哪一段。
4. **发 LOAD**。COMMAND 字先清 0 再置 LOAD，制造干净的上升沿，等回读到 `STATUS_LOADED`。准备完成后 FPGA 处于“装好弹、保险还在”的状态。

prepare 绝不启动序列。它返回后 FPGA 静止，输出维持安全电平。这一点对相机实验至关重要：你可以先 prepare、再武装相机、最后才 fire，三步之间想隔多久都行。

服务端的 `SequencerService.prepare` 还做了一层缓存: 如果新程序的 `sequence_id` 和已准备的完全相同, 就跳过 `prepare_callback` (即跳过整段上传), 直接复用。是否启用由 `ZLC_SEQUENCER_CACHE_PREPARED` 控制。

Python

```
from Zou_lab_control.neutral_atom.devices.sequencer import RemoteSequencer

seq = RemoteSequencer(
    host="192.168.1.50",    # FPGA 计算机的可达地址, 不能是 0.0.0.0
    port=18861,
    channels=["cooling", "repump", "imaging", "mot_coil"],
    clock_hz=50e6,         # 20 ns / tick
)
program = seq.prepare(pulse_table_state)    # 上传边沿表, 但不发射
print(program.sequence_id)                 # 程序内容哈希 (用于去重 / 缓存)
# 此刻 FPGA 装好弹、保险在, 输出仍是安全电平
```

10.4 fire: 一个显式的 start 脉冲, 时序所有权交接

`fire()` 发出一个显式的 **低-高-低** start 脉冲 ($0 \rightarrow 1 \rightarrow 0$)。FPGA 只把**同步后的那个上升沿**当作“开始”。用脉冲而不是电平, 是为了让“开始”这件事在硬件里有一个干净、唯一、可被时钟同步的事件, 不依赖软件维持任何电平。

`fire()` 之后, 正如前面那条铁律: 枪内每条边沿的精确时刻由 FPGA 时钟决定, 软件栈的一切抖动都被隔离在枪外。

服务端 `fire(sequence_payload)` 还有一个安全检查: 如果你传了一个 payload, 它会重新编译并比对 `sequence_id`, 与已 prepare 的不一致就报错——杜绝“准备的是 A, 发射的是 B”。

Python

```
seq.fire()                # 发出 low-high-low start 脉冲, FPGA 开跑
ok = seq.wait_done(timeout=2.0)    # 有限序列: 轮询 STATUS
assert ok, " 序列在 2 秒内没有报告 done"
```

10.5 wait_done: 只对有限序列轮询

`wait_done(timeout)` 轮询 `STATUS` 寄存器 (`DONE` 位), 等一个**有限**序列跑完; 对流式扫描它还会跟随 `CURSOR` 把空出来的 bank 回填下一段。它有一条硬护栏:

Python

```
# SequencerService.wait_done 里:
if program.repeat_forever and timeout is None:
    raise RuntimeError(
        "sequencer wait_done cannot wait forever for a repeat_forever "
        "program; pass a timeout or stop the pulse.")
```

也就是说, 对一个 `repeat_forever=True` 的无限循环程序, `wait_done()` 永远等不到 done, 必须给 timeout 或者主动 stop。这条护栏在客户端的 `PulseController.on_pulse(wait=True)` 里也有对应的、更友好的报错, 提示你三种正确组合。

10.6 safe_state: 回到安全空闲 (Stop Pulse 走这条)

`safe_state()` 把 sequencer 复位到一个安全的空闲态——输出回到安全电平、序列停止。GUI 里的“Stop Pulse”按钮就是调它。在服务端, `set_safe_state()` 还会把 `prepared_program` 清空、状态标为 `safe`, 这意味着下一次必须重新 prepare 才能 fire (不能在 `safe` 之后直接空枪 fire)。

`abort()` 与 `set_safe_state()` 类似, 也会作废已准备的程序并调安全回调, 区别在语义: `abort` 是“出错了, 紧急停”, `safe_state` 是“正常收工, 回到待命”。

10.7 常驻 Vivado 会话: 慢操作只做一次

`VivadoAxisStreamerSession` 是把“慢”集中处理的核心。它在 `start()` 时启动一个 Vivado Tcl 子进程 (`vivado -mode tcl -nolog -nojournal`), 然后跑一段公共初始化 Tcl:

1. 打开硬件目标 (`open_hw_target`, 失败会重试 `-jtag_mode on`)。
2. 加载 `.ltx` 探针文件一次——这让 Vivado 在已烧录的比特流里找到 `jtag_axi` 核 (即 `hw_axi`)。找不到该核会明确报错, 提示当前 FPGA 镜像不是 ZLC pulse-streamer 比特流, 或 `.ltx` 不匹配。
3. 之后整段服务生命周期里复用这同一个进程、同一个已打开的硬件目标、同一个 `hw_axi`。

所以“慢的硬件目标 bring-up 发生在服务器启动时, 而不是第一次 On Pulse 时”。每个后续动作 (`prepare/fire/wait/safe`) 只是往这个活着的进程喂一段带唯一 marker 的 Tcl (一次 AXI 写或读), 读到 marker 即返回。

启动服务器: `warm_start` 让你在第一枪之前就付清启动代价

`run_server(backend="jtag-axi", warm_start=True)` 会在接受任何客户端连接之前就 `hardware_backend.start()`, 把 Vivado 启动 + 打开硬件目标 + 加载探针的几十秒一次性付清。等客户端连上来, 第一次 `prepare` 就已经是热的。

PowerShell

```
# 在 FPGA/Vivado 计算机上, 启动常驻服务 (PowerShell)
$env:ZLC_PS_VIVADO_BIN = "C:\Xilinx\Vivado\2025.1\bin\vivado.bat"
$env:ZLC_PS_VIVADO_LTX = "C:\zlc\zlc_pulse_streamer_top.ltx"
python -m Zou_lab_control.neutral_atom.devices.sequencer_server `
    --backend jtag-axi --host 0.0.0.0 --port 18861 `
    --channels cooling repump imaging mot_coil
# 打印 "Starting persistent Vivado JTAG-to-AXI session before accepting
↳ clients..."
# 慢启动在这里发生; 之后客户端的 prepare/fire 都是热的
```

10.8 流式回填: 扫描点比常驻窗口多时怎么办

扫描窗口在硬件里是一个 **2-bank ping-pong** (`BANK_SIZE=2048` 一个 bank, 两 bank 共 4096 个常驻扫描点)。当扫描点数 N 超过 4096 时, 主机用**流式回填**补齐:

- 引擎依次播放点 $0..N-1$, 用 $(idx/BANK_SIZE) \bmod 2$ 选 bank, 并把已消费点数写进 `CURSOR`。
- 主机读 `CURSOR`, 把游标已经走过的那个 bank 回填下一段 chunk, 并更新 `BANK_READY/BANK*_CHUNK`。

- 引擎只在 `BANK_READY` 为真且该 bank 装的是正确 chunk 时才推进; 回填来晚了就**停住** (hold, 置 `STATUS_UNDERFLOW`), **绝不**播放错误的点。

`repeat_forever` 对一个有限 (但很大) 的流式扫描做反复扫描: fire 时启动一个后台回填线程, 循环把下一段填进空出的 bank; 一次扫描之内是无缝的, 扫描与扫描之间的接缝是一个短暂的安全 hold。

`sequence_id` 缓存让重复 prepare 几乎免费

一次扫描里若某个点和上一点编译出的程序完全一致, `SequencerService.prepare` 通过比对 `sequence_id` (程序内容哈希) 直接跳过整段上传, 第二次 `On Pulse` 只是几次控制写。而扫描点本身在硬件里**无缝迭代** (仿射 slot 向量 + 2-bank 窗口), 并不需要逐点重新 prepare。

10.9 RemoteSequencer 与 RPyC: 客户端代理

`RemoteSequencer` 是控制计算机侧的瘦代理。它本身不碰硬件, 只把请求序列化成 JSON 经 RPyC 发给服务端。

连接 `open()` 建立 RPyC 连接 (`rpyc.connect`, 可选 `ssl_connect`), 并立即调远端 `snapshot()`, 用服务端的 `channels/clock/trigger_channels` 覆盖本地——**服务器是通道顺序与时钟的唯一真相来源**。`sync_request_timeout=None` 让长动作 (如硬件 prepare) 不会被客户端超时打断。

host 护栏 构造时如果 host 是空、`0.0.0.0` 或 `::` 会直接报错——这些是“监听所有接口”的服务端地址, 不是客户端能连的地址。

传输 `prepare/fire` 把 `PulseSequence/PulseTableState` 经 `timing_payload_to_dict` 转 dict, `json.dumps` 后发出; `prepare` 收回的程序 JSON 反序列化为 `RuntimeSequenceProgram` 存进 `\_last_program`。

Python

```
seq = RemoteSequencer(host="192.168.1.50", port=18861,
                      channels=["cooling", "repump", "imaging", "mot_coil"],
                      clock_hz=50e6, connect_on_init=True)
print(seq.snapshot()["remote"]["state"])    # 远端状态机当前态
seq.set_safe_state()                       # 远端回到安全空闲
```

10.10 五个 Sequencer 类各司其职

仓库里有五个实现 `SequencerDevice` 契约的 sequencer, 角色互不重叠:

VirtualSequencer 纯软件假人。无硬件, 用于离线开发与单元测试, 验证编译/契约而不接 FPGA。

RuntimeSequencer 本地适配器: 直接内嵌一个 `SequencerService`, 在**同一进程**里走完整的 `prepare/fire/wait` 协议 (默认无硬件回调, 用 `sleep_scale` 模拟时长)。适合在一台机器上端到端联调协议。

ManualSequencer 手动/逐步模式, 给需要人工介入或单步推进的台架调试用。

RemoteSequencer RPyC 客户端代理, 连远端真实硬件 (本章主角)。

VerilogSequencer 走 `verilog.py` 那条路, 把脉冲编译成边沿表/HDL 工件 (`VerilogBuild`), 用于生成/检视而非实时驱动。

10.11 repeat_forever 的两种用法：示波器 vs. 相机

repeat_forever 这个旗标的取舍直接决定实验对不对：

repeat_forever=True (无限自由跑) 适合**示波器调参**。让 FPGA 不停重复同一序列，你在示波器上实时看波形、转旋钮、立刻看到变化。注意此时 wait_done() 永远不会返回 done (见前面的护栏)，要用 on_pulse(wait=False, repeat_forever=True)，靠 Stop Pulse 收尾。

repeat_forever=False (有限) 适合**相机采集**。正确做法：**先武装相机 (arm)**，**再生成恰好需要的有限触发序列**，让相机在确定的触发数下读出。

千万不要让 qCMOS 去等一个无限自由跑的循环

有限相机采集必须用有限序列。如果让 qCMOS 等在一个 repeat_forever=True 的自由跑循环上，相机不知道何时该停、采了几帧，会拿到错配甚至无限挂起的读出。正确顺序是：arm 相机 → prepare 有限序列 → fire → wait_done(有限) → 读图。

Python

```
# 一次有限相机采集的完整循环（控制计算机）
camera.arm(n_frames=expected_triggers)          # 1) 先武装相机
seq.prepare(finite_pulse_with_camera_triggers) # 2) 上传恰好需要的有限触发序列
seq.fire()                                       # 3) start 上升沿，FPGA
↪ 接管纳秒时序
ok = seq.wait_done(timeout=expected_duration_s + 0.5) # 4) 轮询 done
assert ok
frames = camera.read()                          # 5) 读出
seq.set_safe_state()                            # 6) 回到安全空闲
```

10.12 从零跑一次真实实验：完整清单

把以上各层串起来，一个新人在两台机器上跑出第一枪的步骤：

1. **FPGA 计算机**: 跑 `fpga\textbackslash build\_and\_program.bat(create_project.tcl` 生成工程、编译并烧录 `zlc_pulse_streamer_top.bit` 与配套 `.ltx`)。 `.ltx` 必须与比特流来自同一次编译，否则 server 找不到 `jtag_axi` 核。
2. **FPGA 计算机**: 设置 `ZLC_PS_VIVADO_BIN` 与 `ZLC_PS_VIVADO_LTX`, `run_server(backend="jtag-axi", warm_start=True, host="0.0.0.0", port=18861, channels=...)`。等它打印完常驻会话启动信息——慢启动在此付清。
3. **控制计算机**: 构造 `RemoteSequencer(host=<FPGA 机 IP>, port=18861, channels=..., clock_hz=50e6)`, `open()` 自动用远端 snapshot 校准通道与时钟。
4. 用 GUI 或脚本做出脉冲表 (`PulseTableState`)，需要扫描就绑 slot、填 `scan_table`。
5. 示波器调参阶段: `on_pulse(wait=False, repeat_forever=True)` 自由跑，看波形拧旋钮。
6. 正式采集阶段: arm 相机 → `prepare(有限序列)` → `fire()` → `wait_done(timeout)` → 读图。扫描的后续点靠差分上传，几乎免费。
7. 收工: `set_safe_state()` (GUI 的 Stop Pulse)，输出回安全电平、作废已准备程序。

一句话记住整章

慢操作（Vivado/JTAG/探针）在**服务器启动**时做一次；prepare 在 reset 拉高、输出钳零的情况下用差分上传写边沿表，绝不发射；fire 给一个 start 上升沿，把纳秒时序的所有权**交接**给 FPGA 时钟；wait_done 只对有限序列有意义；相机采集必须用有限序列、先武装后发射。

11

N-slot 扫描模型与硬件无缝扫描（深入）

本章把“扫描”这件事从头讲透。读完之后，一个新人应该能在 Notebook 里绑定可扫字段、装入扫描表、编译成一个 FPGA 程序，并理解为什么一个一百万点的扫描**不会**把边沿表撑大、为什么 DAC 值与周期长度可以**在一次连续运行里同时扫描**。（只有 **duration** 和 **dac** 两类可扫；channel 延时不是扫描量，它是一个固定的、事件调度的逐信号输出延时（32 位范围），与任意扫描共存——见上一章“固定延时如何与扫描共存”。）

11.1 为什么是“命名 slot”而不是 x/y

扫描引擎用任意多个**命名 slot**：**s0**、**s1**、**s2**、.....每个 slot 是一个独立的扫描维度（**没有 x/y 二轴的概念**）。

一个 slot 由三件东西定义：

kind（种类） 它绑定的是哪一类字段。只有**两种**合法值（`SCAN_SLOT_KINDS = ("duration", "dac")`）：**"duration"**（某个周期的时长）、**"dac"**（某个周期里、某条模拟总线上的 DAC 输出值）。**channel 延时不在其中**——它是每条 channel 的固定值，`bind_field("delay", ...)`会报错。

target（目标） 在该种类里具体指哪一个字段。对 **duration** 是周期下标（字符串化的整数，如 **"1"**）；对 **dac** 是 **"<bus>@<period_index>"** 形式，例如 **"da_dipole@2"**（第 2 个周期、da_dipole 总线上的值）。

unit / nominal（单位与标称值） 时间类 slot 带单位（**ns/us/ms/s**）；**dac** slot 的“单位”是**原始 10 位 DAC 码**，没有电压换算。**nominal** 是绑定那一刻字段的当前值，作为默认/回填值。

slot 的编号就是它的 **s** 变量：第 **i** 个 slot 在所有表达式里就是字符串 **s\{i\}**（`slot\_var(0) = "s0"`）。这一点很重要——**slot 的序号即变量名**，所以删除一个 slot 会让后面的 slot 全部重新编号（见 `unbind\_slot`）。

11.2 绑定字段：bind_field 与 scan dot

把一个普通字段变成“可扫描的符号字段”，叫做**绑定**。绑定后，该字段在内部不再是一个数字，而是被改写成符号 **s0/s1/.....**，参与表达式求值。

两种绑定途径，效果完全一致：

- **GUI**: 点击字段右侧那个嵌入的小圆点 (scan dot)。圆点把字段染成淡橙底、深橙字, 并标上 slot 编号 (橘色); 字段随即被禁用为符号。再点一次解除。
- **API**: 调用 `state.bind_field(kind, target)`, 返回新 slot 的整数下标。

状态的所有权很清晰: `Zou\_lab\_control/neutral\_atom/timing/pulse\_table.py` 里的 `PulseTableState` 持有 `scan\_slots` (有序的 `ScanSlot` 列表) 和 `scan\_table` (数据)。GUI 只是这个状态的视图; 编译器只读这个状态。

`bind\_field` 的关键语义:

- **幂等**: 重复绑定同一个 `(kind, target)` 返回已有下标, 不会新建 slot (所以 GUI 里重复点 dot 不会清零或重排编号——编号保留)。
- 绑定时若不给 `nominal`, 会从字段当前值自动读取 (`\_read\_field\_nominal`)。
- 绑定后会该字段改写成符号: `duration` → 周期时长变成 `"s0"`; `dac` → 该周期该总线段变成 `\{"mode": "edge", "value": "s0"\}`。(`delay` 不能绑定——它是固定值, `bind_field("delay", ...)` 抛 `ValueError`。)
- 已有的扫描表每一行会自动追加一列 (用 `nominal` 填充), 保持 `scan\_table` 列数 = slot 数。

Python

```
from Zou_lab_control.neutral_atom.timing.pulse_table import
↳ PulseTableState

state = PulseTableState.demo() # 任意已搭好的脉冲表

# 1) 扫第 1 个周期的时长 (按 s0)
s0 = state.bind_field("duration", "1", unit="us") # 返回 0

# 2) 扫第 2 个周期里 da_dipole 总线上的 DAC 值 (按 s1)
s1 = state.bind_field("dac", "da_dipole@2") # 返回 1

# cooling 的延时是一个固定的每通道值, 不进扫描表 (也不能绑定)
state.delays["cooling"] = 500.0 # 固定 500 ns
↳ 输出延时
# state.bind_field("delay", "cooling") # -> ValueError: delay is a fixed
↳ per-channel value

print(state.scan_slots) # 两个 ScanSlot, kind 分别是 duration/dac
```

s 变量只能引用 slot

时间表达式里只能出现 `s0 .. s\{N-1\}`, 没有 `x/y` 也没有别的自由符号。一个时长可以写成依赖多个 slot 的仿射表达式 (如 `"s0 + 2*s1"`), 但必须是 slot 的一次线性组合——见后文“限制”。

11.2.1 解绑与重新编号

`state.unbind\_slot(index, restore=...)` 删除一个 slot。因为**编号即变量名**, 删除中间 slot 会把它后面的所有 slot 往前移: 原来的 `s2` 变成 `s1`, 依此类推, 所有引用它们的字段会被自动重写到新编号 (`\_renumber\_slot\_bindings`)。 `restore` 给出解绑后字段恢复成的字面值 (不给则时长回到 1000 ns、DAC 回到 hold)。

11.3 scan_table: N_points × N_slots 的数据矩阵

扫描的“数据”和扫描的“结构”是分开的。结构是 `scan\_slots` (哪些字段被扫); 数据是 `scan\_table`: 一个 **N_points 行 × N_slots 列** 的二维数组。

- **行** = 一个扫描点 (一次相机曝光对应一行)。
- **列 j** = slot `s\{j\}` 在该点的取值, **用该 slot 的显示单位**: 时间类 slot 用 ns (或绑定时的单位, 内部按 `UNITS\_T0\_NS` 折算成 ns), `dac` 类用**带符号 DAC 值** (-512..+511, 0 = 0 V; 线上自动 +512 变 offset-binary 码)。

装入数据有几种方式:

从文件 `state.load\_scan\_table(path)` 支持 `.npy` / `.csv` / `.txt`。文件里就是一个 $N \times N_slots$ 的矩阵, 列顺序必须和 slot 顺序一致。

直接给数组 `state.set\_scan\_table(rows)`, 会按当前 slot 数归一化 (补齐/截断列)。

GUI Scan 页 在 Scan 标签页里用代码生成参数 (每个 slot 一段 Python), 或点 Load 从文件载入。

Python

```
import numpy as np

# 两列对应 s0(us 周期时长)、s1(DAC 码)
table = np.array([
    [10.0, 0],      # 点 0: 周期 =10us, DAC=0
    [20.0, 256],    # 点 1
    [30.0, 768],    # 点 2
    [40.0, 511],    # 点 3: DAC 正满量程
])
state.set_scan_table(table)

# 或从文件
# state.load_scan_table("scan_params.npy")
```

单位约定再强调一次

时间列写你看到的数 (ns/us/ms/s 取决于该 slot 的 unit); DAC 列写**带符号 10 位值 -512..+511** (0 = 0 V, +511 正满量程、-512 负满量程)。电压到 LSB 的换算由你在写表前自己完成; 线上 offset-binary 码由编译器自动 +512 生成。

11.4 主机如何编译: 一个仿射模板 + 一张流式点表

这是整套设计的核心, 也是它省 LUT 的根本原因。主机**不**为每个扫描点各生成一份边沿表。它只生成两样东西:

1. **一份仿射边沿模板**: 所有边沿的“基准 tick”加上每个边沿对每个 slot 的系数。
2. **一张流式扫描点表**: 就是 `scan\_table` 转成整数码 (`scan\_points`)。

运行时, FPGA 对每个扫描点用同一份模板, 按下式实时算出每条边沿的有效 tick:

text

```
effective_tick = base_tick + (Σ_j coeff_j * slot_j) >> COEFF_FRAC_BITS
```

其中 `COEFF\_FRAC\_BITS = 8` (即 `scan\_coeff\_frac\_bits`)。系数用 8 位定点小数表示，所以斜率可以是非整数。`slot\_j` 是当前扫描点里 slot j 的整数值（时间类已折算成 tick 量纲，dac 类是 DAC 码）。

由此得到本设计最重要的性质：

大扫描不会撑大边沿表

边沿表的长度只由**模板复杂度**（有多少条边沿、多少周期/通道）决定，**与扫描点数无关**。扫 10 个点和扫 100 万个点，边沿模板一模一样；多出来的只是流式点表里多几行整数。存储是 $O(\text{edges}) + O(\text{points} \times \text{slots})$ 。

编译入口是 `state.compile\_scan(clock\_hz=...)` (薄封装)，它转调 `Zou\_lab\_control/neutral\_atom/device` 里的 `compile\_pulse\_table\_scan\_runtime\_program`，返回一个 `RuntimeSequenceProgram`。该程序对象携带：

`slot_count / slot_kinds` slot 数量与每个 slot 的 kind。

`tick_slot_coeffs` 每条边沿对每个 slot 的系数（行 = 边沿，列 = slot）。

`loop_end_slot_coeffs` 循环结束 tick 对各 slot 的系数（让循环边界也随扫描平移）。

`scan_points` 整数化的扫描点表 (`list[list[int]]`)。

`scan_coeff_frac_bits` 即 8。

`ticks / masks` 基准边沿模板（绝对 tick + 62 位通道掩码）。

`bus_segments` 模拟总线段（含 DAC 无缝扫描所需的 `value_select` 与段 tick 系数）。

Python

```
program = state.compile_scan(clock_hz=50e6)    # 50 MHz → 20 ns/tick

print(program.slot_count)                      # 2
print(program.slot_kinds)                      # ['duration', 'dac']
print(program.scan_coeff_frac_bits)            # 8
print(len(program.scan_points))                # 4 (点数)
print(program.tick_slot_coeffs)                # 每条边沿 × 2 个系数
print(program.channel_delays)                  # 固定延时作为常数携带 (不在 slot 里)
```

FPGA 拿到这个程序后，**无缝地**遍历所有扫描点：每点对应模板的一次实例化，每点发**一次相机触发**，点与点之间不停机。这就是“硬件无缝扫描”。

11.5 DAC 值 + 时长：同时扫两者（并叠加一个固定延时）

这是相对早期版本最大的突破。早期 DAC 值的扫描和周期时长的扫描是互斥的（怕段的 tick 不跟着时长平移而失步）。现在不互斥了：**DAC 值与周期时长可以在同一次无缝运行里同时扫描**。再叠加一个**固定的**通道延时也完全没问题——延时不是扫描量，它作为常数携带，作为一条延时线加在输出上、**与两类扫描均匀组合**。

实现的两个支柱：

1. **段 tick 也仿射发射**：每个模拟总线段的起止 tick 不再是固定值，而是 `base + 系数 × slot`，用和边沿同一套 `effective\_tick` 公式。于是当你扫一个周期的时长时，落在该周期后面的 DAC 段会**自动跟着平移**，不会失步。

2. **value_select 选值来源**：每个总线段带一个选择位。0 表示用字面值；j+1 表示运行时从 scan slot j 读取低位作为 DAC 码。绑定 kind="dac" 时，编译器把对应段的 value_select 指向那个 slot。

绑定 DAC 用 `target = "<bus>@<period_index>"`，扫描表对应列写带符号 10 位 DAC 值 (-512..+511, 0 = 0 V)。下面是一个同时扫 duration 与 dac、并给一条 channel 设固定延时的可运行端到端例子：

Python

```
import numpy as np
from Zou_lab_control.neutral_atom.timing.pulse_table import
↳ PulseTableState

state = PulseTableState.demo()

# --- 同时绑定两个可扫维度 ---
s0 = state.bind_field("duration", "1", unit="us")    # s0: 第 1 周期时长 (us)
s1 = state.bind_field("dac", "da_dipole@2")          # s1: 周期2 上 da_dipole
↳ 的 DAC 值

# --- 一个固定的（不可扫）每通道延时，叠加在输出上 ---
state.delays["cooling"] = 150.0                      # 固定 150 ns 延时
↳ （事件调度，上限 32 位±42.9s）

# --- 扫描表：4 个点，列顺序 = s0(us), s1(DAC 码) ---
state.set_scan_table(np.array([
    [10.0, 0],
    [20.0, 256],
    [30.0, 768],
    [40.0, 511],
]))

# --- 一次编译，得到一个无缝扫描程序 ---
program = state.compile_scan(clock_hz=50e6)

assert program.slot_count == 2
assert program.slot_kinds == ["duration", "dac"]
assert len(program.scan_points) == 4
# DAC 段带 value_select 指向 s1; 该段 tick 随 s0 平移（仿射系数非零）
# cooling 的固定延时作为常数落在 program.channel_delays（不在任何 slot 里）
```

运行这个程序时，FPGA 对四个点依次实例化模板：每点周期 1 的时长、周期 2 上 da_dipole 的输出码都按该行取值，DAC 段的位置随时长平移；cooling 的固定延时在每一点都把它的输出整体推后 150 ns（事件调度，32 位范围），每点打一次相机触发——全程不停机。

谁拥有什么状态：生命周期一览

- `PulseTableState` (pulse_table.py) 拥有 `scan\_slots` + `scan\_table` + 脉冲结构。GUI 是它的视图。
- `compile\_scan` / `compile\_pulse\_table\_scan\_runtime\_program` (sequencer.py) 把状态只读编译成 `RuntimeSequenceProgram`（仿射模板 + 整数点表 + 总线段）。

- RTL(`fpga/pulse\_streamer/zlc\_edge\_streamer.v`)按 `effective\_tick` 公式实时实例化模板, 无缝遍历 `scan\_points`, 每点一相机触发; 固定延时由引擎作为逐通道输出延时线施加。

11.6 限制与编译器拒绝的情形

仿射模板 + 流式点表很强, 但有它换来省 LUT 的代价。编译器会主动拒绝违反前提的输入——这是好事, 它在主机端就把会失步的程序挡下来:

时间表达式必须仿射 所有 duration 表达式必须是 slot 的一次线性组合 ($s_0 + 2*s_1$ 可以, s_0*s_1 或 s_0*s_2 不行)。非仿射的时序会被拒绝。(延时不参与扫描表达式——它是固定值。)

边沿顺序不能在扫描中翻转 如果某个扫描点会让两条边沿的先后顺序交换 (模板的边沿排序在不同点之间不一致), 编译器拒绝。换句话说, 整段扫描里边沿的相对顺序必须稳定。

ramp 端点可以被扫描 DAC 无缝扫描对 **edge 与 ramp 一视同仁**: 把 ramp 周期的值绑定到扫描槽 (GUI 里点该周期的扫描圆点, 周期**保持 ramp**), 硬件就在每个扫描点把斜坡从携带入值斜变到**该点的码**——段的两个端点各有独立的运行时选择器 (`value_select/stop_value_select`), 所以「从被扫电平出发的 ramp」与「斜向被扫电平的 ramp」都无缝支持 (真引擎 `xsimsim/tb\_ramp\_scan.v` 逐 tick 验证 Bresenham 阶梯)。唯一新增的约束: **被延时**总线上的长 ramp 受逐 bit 事件 FIFO 容量约束 ($\min(\text{延时}, \text{斜坡长}, \text{摆幅}) \leq \text{bus_evt_fifo_depth}$, 被扫端点按满摆幅保守计), 超出在编译时清晰报错。

DAC 列是码不是电压 再说一遍: `dac` slot 的扫描表列是带符号 10 位值 -512..+511 ($0 = 0\text{ V}$), 线上才是 offset-binary 码。

单点 Notebook 扫描的便捷写法

如果你只想一发设一个值 (不走硬件无缝遍历), 用 `state.with\_slots\_resolved(\{"s0": 20.0, "s2": 768\}\})`: 它返回一个**非扫描**副本, 把每个 slot 替换成常量 (时间类给 ns, dac 给整数码), `scan\_slots` 和 `scan\_table` 清空。适合在循环里逐点手动设值的简易扫描。

12

真实硬件 Runbook (深入)

本章是一份“从零到完成一次真实成像”的完整操作手册。它面向第一次接触本系统的人：把整套控制链路拆成几层，逐层说明谁拥有什么状态、什么时候做什么、命令长什么样、出问题时往哪里看。读完本章并照做，你应当能在两台真实计算机上把 FPGA 编译、烧写、起服务、从 notebook 连上去打一次相机成像脉冲。

12.1 系统全景：两台计算机、四个层次

整套系统跑在**两台物理计算机**上，中间用 RPyC 网络连接：

控制/qCMOS 计算机 跑实验 notebook 和相机。这台机器上 `na.connect(...)` 创建一个 `NeutralAtomSession` 会话对象；它管理相机、trap array、sitemap、以及一个**远程** sequencer 句柄。这台机器**不需要**装 Vivado。

FPGA/Vivado 计算机 物理上连着 Artix-7 (xc7a35tfgg484-2) 开发板和 Vivado。这台机器跑两件事：一次性的**编译 + 烧写** (`fpga\textbackslash build\_and\_program.bat`)，以及常驻的**脉冲流服务器** (`fpga\textbackslash run\_server.bat`)。服务器内部持有一个常驻 Vivado `hw_axi` 会话，通过 JTAG-to-AXI 把脉冲程序映像写进 FPGA 的 BRAM。

从上到下，请把控制链路理解成四层，每层有明确的**状态所有者**：

1. **Notebook / Session 层** (控制机)： `exp = na.connect(...)` 返回的会话对象。它拥有“当前序列” (`exp.sequence`)、相机配置、标定结果。
2. **RemoteSequencer / RPyC 层**：会话里的 sequencer 句柄是一个网络代理。它把 `prepare/fire/wait\_done/safe\_stop` 这些动作通过 RPyC 发到 FPGA 机的 `SequencerService`。
3. **Server / Vivado 会话层** (FPGA 机)： `run\_server.bat` 起的进程持有一个常驻 Vivado `hw_axi` 会话 (jtag-axi 后端)，是真正会经 JTAG-to-AXI 写 BRAM、改 FPGA 内存的人。它拥有“已经烧进去的 bit 文件”和“当前已载入的程序”。
4. **硬件层**：FPGA 里跑的 `zlc\_pulse\_streamer\_top` (边沿表 + 1-tick 预取 + 2-bank 流式扫描 + 仿射扫描引擎 + 模拟总线)，以及 62 路物理输出。它拥有“正在跑的脉冲”和真实的电平。

一句话记忆

Notebook 说“我要打这个序列”，Session 把它编译成边沿表/扫描表通过 RPyC 发给 FPGA 机的 Vivado 会话，Vivado 会话把它打包成 BRAM image 经 JTAG-to-AXI 写进 FPGA，FPGA 自己按时钟把电平打出来。相机的外部触发也是这 62 路里的一根线。

路径长度：一定要短

把整个仓库放在**很短的路径**下，例如 `D:\textbackslash ZLC`。原因是 Vivado 2019 的 debug-core 会在工程目录里生成很深的临时路径，而 Windows 对路径长度敏感；仓库放得太深会让综合/实现在写 debug-core 临时文件时莫名失败。这不是建议，是硬约束——默认工程目录是 `fpga\textbackslash build\textbackslash pulse\_streamer`，再加上仓库本身的深度，很容易超界。

12.2 第一步：在两台机器上安装

两台机器都要装。安装脚本是 `install\_requirements.bat`，它做四件事：把 `requirements.txt` 装进**当前选中的解释器**、用 `pip install -e .` 以可编辑方式装上本仓库、注册一个 Jupyter kernel、并把这个解释器的完整路径**记到** `.zlc\_python\_path` 文件里。

PowerShell

```
# 在仓库根目录（例如 D:\ZLC）打开 PowerShell，两台机器都要做一次
cd D:\ZLC

# 方式 A：让脚本自动从 VSCode/Jupyter kernel 推断解释器
.\install_requirements.bat

# 方式 B：直接指定你要用的 python.exe（最稳）
.\install_requirements.bat C:\Users\you\miniconda3\envs\zlc\python.exe
```

记住 `.zlc\_python\_path` 这个文件的作用：之后 `pulse\_gui.bat`（脉冲编辑器 GUI）和 `fpga\textbackslash run\_server.bat`（服务器）都会**优先读取同一个解释器**，这样保证 GUI、服务器、notebook 用的是**同一套依赖**。这是避免“我这边能跑、那边报 ImportError”的关键设计。

控制/qCMOS 机 装完后，在 VSCode 里选 kernel “Python (Zou lab control)”，重启 Jupyter kernel，就可以跑 notebook。

FPGA/Vivado 机 装完后还要确认 Vivado 能被找到。脚本会在 `C:\textbackslash Xilinx\textbackslash Vivado\textbackslash < 版本 >\textbackslash bin\textbackslash vivado.bat` 和 `D:\textbackslash Xilinx\textbackslash ...` 下自动探测多个版本。如果你的 Vivado 不在默认路径，显式指定：

PowerShell

```
# 仅当自动探测找不到 Vivado 时才需要设置（指向 vivado.bat）
$env:ZLC_PS_VIVADO_BIN = "C:\Xilinx\Vivado\2019.2\bin\vivado.bat"
```

12.3 第二步（FPGA 机）：编译 + 烧写

所有 FPGA 侧编译/烧写都走一个入口：`fpga\textbackslash build\_and\_program.bat`。它有几个模式：

--check 离线自检：不连硬件、不用真实引脚，只做 HDL 综合 + 容量自查。第一次部署、或改过

RTL/Tcl 之后，先跑这个。它核对 top 是 62 路 wrapper，BRAM image 几何（边沿/扫描/总线区基址）和 `host.image.solve_capacity` 在 35T 上解出的 4096 边沿 + 4096 常驻扫描点 + 流式一致。

(无参数) 编译并烧写：跑 create-project Tcl 生成工程、综合、实现、生成 bit/ltx，然后烧进 FPGA。

--build-only 只编译，不烧。

--program-only 用已有的 bit/ltx 直接烧。

--diagnose 列出 Vivado 看得到的硬件 target/device，确认 JTAG 线缆连上了。

PowerShell

```
cd D:\ZLC

# 1) 先离线自检: HDL 综合 + 容量
.\fpga\build_and_program.bat --check

# 2) 自检过了，连上板子，确认能看到硬件
.\fpga\build_and_program.bat --diagnose

# 3) 完整编译 + 烧写（耗时较长，是真综合 + 实现）
.\fpga\build_and_program.bat
```

默认工程落在 `fpga\textbackslash build\textbackslash pulse\_streamer`；编译产物 **bit 文件**和 **ltx 探针文件**在它的 `pulse\_streamer.runs\textbackslash impl\_1` 子目录下，文件名是 `zlc\_pulse\_streamer\_top.bit`和 `zlc\_pulse\_streamer\_top.ltx`。

XDC 引脚映射与 .ltx 探针文件

默认 XDC 是 address-switch 板的引脚映射 (`fpga\textbackslash board\_config\textbackslash board.xdc`, 见 `fpga/board\_config/README.md`)。换板子/换线缆映射时用 `$env:ZLC_PS_XDC` 覆盖。create-project Tcl 会生成 `jtag_axi + axi_bram_ctrl` + BRAM 这套数据通路。这一点很关键：服务器是靠 **JTAG-to-AXI 写 BRAM 控制 FPGA** 的，所以服务器加载的 `.ltx` 必须和烧进去的 bit 是**同一次编译**产生的，否则找不到 `jtag_axi` 核。

12.4 相机成像的物理输出（必须记住的四根线）

成像序列只用到四路物理输出，请把这四条对应关系刻在脑子里——它们既是 XDC 里的引脚，也是 notebook 里 `configure_imaging` 会驱动通道：

trap (光阱) `ch09` → 引脚 **M17**

cooling (冷却) `ch00` → 引脚 **F15**

probe (探测) `ch03` → 引脚 **N15**

emCCD / 相机外触发 `ch11` → 引脚 **M13** (这根线就是 qCMOS 的外部触发)

Session 在编译成像序列时会自动做这个映射：它检查 sequencer 报上来的通道里是否同时含有 `ch00/ch03/ch09` 且有触发通道，若是就把 `trap/cooling/probe/trigger` 绑到 `ch09/ch00/ch03`/触发通道。所以你在 notebook 里写 `configure_imaging()`，底层就是在这四根线上排队脉冲。

时钟陷阱: 50 MHz / 20 ns

默认时钟是 **50 MHz**, 最小步进 **20 ns** (一个 tick)。如果你在示波器上看到脉冲长度**正好是设定值的两倍**, 几乎可以肯定是某一层还在按**旧的 100 MHz**假设算 tick。逐层检查 control (notebook 传的 `clock-hz`)、server (`run\_server.bat` 的 `ZLC_PS_CLOCK_HZ`)、GUI (编辑器里的时钟设置) 三处是否都是 50 MHz。三处必须一致。

12.5 第三步 (FPGA 机): 启动脉冲流服务器

烧好 bit 之后, 在 FPGA 机上常驻一个服务器进程, 它就是 RPyC 的服务端。入口是 `fpga\textbackslashrun\_server.bat`。

先**干跑一次配置检查** (不真正起服务, 只打印它将要用的全部配置), 确认 bit/ltx/XDC/通道/触发/时钟/容量都对:

PowerShell

```
cd D:\ZLC
.\fpga\run_server.bat --check-config
```

它会打印类似这样的清单 (请逐行核对):

text

```
Host:      0.0.0.0:18861
Backend:   jtag-axi
Project:   ...\fpga\build\ps
Bit:       ...\impl_1\zlc_pulse_streamer_top.bit
LTX:       ...\impl_1\zlc_pulse_streamer_top.ltx
Channels:  ch00 ch01 ... ch61
Clock:     500000000 Hz
Capacity:  max_edges=4096 bank_size=2048 (4096 resident) + UNBOUNDED
↪ streaming
```

确认无误后, 正式起服务:

PowerShell

```
.\fpga\run_server.bat
```

默认它在 **host 0.0.0.0**、**port 18861** 上监听, 后端是 **jtag-axi** (持久 Vivado `hw_axi` 会话)。窗口会一直挂着——这是预期行为, **不要关掉它**。常用环境变量覆盖:

ZLC_PS_HOST / ZLC_PS_PORT 监听地址/端口, 默认 `0.0.0.0 / 18861`。

ZLC_PS_CLOCK_HZ 时钟, 默认 `500000000`。

ZLC_PS_XDC 覆盖引脚映射 XDC (通道名/触发通道是从 XDC 推断的)。

ZLC_PS_VIVADO_BIN Vivado 不在默认路径时指定 `vivado.bat`。

ZLC_FPGA_SERVER_PYTHON 强制服务器用某个 python.exe (否则读 `.zlc\_python\_path`)。

server 启动失败最常见的原因

`run\_server.bat` 在起服务前会硬性要求找到 **.ltx 探针文件** (server 靠它在比特流里定位 `jtag_axi` 核)。如果报 “no Vivado .ltx probe file was found” 或 “.ltx ... does not exist”, 说明你还没烧 bit、或者 bit 和 ltx 不在它找的默认 `impl_1` 路径下。解决: 先跑

`build\_and\_program.bat`, 或用 `$env:ZLC_PS_VIVADO_LTX` 指到 Vivado Program Device 时实际用的那个 Probes 文件。

12.6 第四步 (控制机): 从 notebook 连上去

服务器跑起来后, 回到**控制/qCMOS 机**的 notebook。连接用 `na.connect`, 第一个位置参数是**配置名** (真实硬件用 `"remote_template"`), 用 `sequencer={...}` 传远程地址, `open_devices=True` 让它立刻打开相机等真实设备。

Python

```
import Zou_lab_control.neutral_atom as na

exp = na.connect(
    "remote_template",
    sequencer={"host": "192.168.0.42", "port": 18861}, # FPGA 机的地址/端口
    open_devices=True,
)
```

`exp` 就是 Session 对象, 它拥有当前实验的所有状态。接下来配置成像序列、做飞行前检查、再真正打:

Python

```
# 1) 配置一次成像序列。默认 trigger_width=20us, pre_trigger=100us。
#     exposure 不传则用相机当前曝光; load=True 表示序列里包含装载阶段。
seq = exp.timing.configure_imaging(exposure=20e-3)

# 2) 飞行前检查: 编译 verilog、做安全/容量校验, 并在失败时直接抛异常。
exp.timing.preflight().raise_if_failed()

# 3) 看一眼将要打的序列 (可选, 画在 notebook 里)
exp.timing.plot_sequence()
```

`configure_imaging` 的参数都是关键字参数:

exposure 相机曝光 (秒)。传了就同时把相机 `configure(exposure=...)`, 不传则沿用相机当前值。

load 是否在序列里包含原子装载阶段, 默认 `True`。

trigger_width 给相机的外触发脉冲宽度, 默认 `20e-6` (20 us)。这就是 emCCD/ch11/M13 那根线上的脉冲。

pre_trigger 触发前的提前量, 默认 `100e-6` (100 us)。

`preflight()` 返回一个 **PreflightReport**: 它带 `ok`、`errors`、`warnings`、序列表、设备快照、以及编译出的 verilog 信息。`raise_if_failed()` 在 `ok` 为假时把所有 error 拼成异常抛出——**务必**在真正打硬件前调它。想看细节可以 `print(exp.timing.preflight().summary())`。

完成检查后, 就可以真正采集 / 读出 (具体的 capture/readout API 见设备手册与主手册对应章节, 这里强调的是它们**此时**才会触发真实硬件动作)。一次最小的“连接到打出脉冲”流程到此就闭环了。

一次完整闭环的状态流转

`configure_imaging` 在 Session 里生成 `exp.sequence` (控制机内存) → `preflight` 把它编译成边沿表/扫描表 (仍在控制机) → 采集时通过 RemoteSequencer 经 RPyC 把表发到 FPGA 机的 Vivado 会话 → Vivado 会话 `prepare` (保持复位、用 `prog_we` 翻转逐行写入、释放) → `fire` (启动脉冲) → `wait\done` (轮询 done) → `safe\_state`。相机在 emCCD 那根线上被同一份序列触发, 图像回到控制机。

12.7 排错手册 (按现象查)

- 现象: 示波器上脉冲长度正好是设定值的两倍** 时钟假设错了。某层还按 100 MHz 算 tick, 而硬件是 50 MHz (20 ns/tick)。逐一核对: notebook 连接/序列的时钟、`run\_server.bat` 的 `ZLC_PS_CLOCK_HZ` (应为 50000000)、GUI 编辑器里的时钟。三处必须都是 50 MHz。
- 现象: 综合/实现在写 debug-core 临时文件时失败, 路径报错** 仓库放得太深。把整个仓库移到很短的路径 (如 `D:\textbackslashlash ZLC`) 重试。Vivado 2019 的 debug-core 临时路径对长度敏感。
- 现象: run_server.bat 报找不到.ltx 探针文件** 还没烧 bit, 或 bit/ltx 不在默认 `impl\_1` 下。先 `build\_and\_program.bat` (或 `--program-only`), 或用 `$env:ZLC_PS_VIVADO_LTX` 指向实际烧写时用的 Probes 文件。务必保证 ltx 和 bit 来自同一次编译。
- 现象: 找不到 Vivado** 自动探测覆盖 2019.1–2025.2 的默认 `C:/D:` 路径。不在其中就设 `$env:ZLC_PS_VIVADO_BIN` 指向 `vivado.bat`。
- 现象: notebook 报 ImportError, 但 GUI/服务器都正常** 三处用的解释器不是同一个。重跑 `install\_requirements.bat` 把仓库装进 notebook 选中的那个 kernel, 并确认 `.zlc\_python\_path` 指向同一解释器。`pulse\_gui.bat` 和 `run\_server.bat` 都会优先读这个文件。
- 现象: HDL 综合 / 容量自检失败** RTL、top、image 几何漂移了。`build\_and\_program.bat --{-}check` 会报哪个 BRAM 区段或宽度对不上 `host.image.solve_capacity` 解出的契约 (边沿 4096、扫描 bank 2048、mask=62、tick=32)。`host.image` 是单一真相源——RTL `localparam` 和 `create-project Tcl` 都从它派生, 改回契约值后重新 `create-project`。
- 现象: 连不上 FPGA 机 / RPyC 超时** 确认 `run\_server.bat` 窗口还开着且在监听 `0.0.0.0:18861`; notebook 的 `sequencer={"host": ..., "port": 18861}` 里 host 是 FPGA 机的真实 IP; 防火墙放行 18861。先在 FPGA 机上 `--check-config` 确认它确实起在期望端口。
- 现象: 相机不触发, 但其它脉冲正常** 检查 emCCD 那根线: 通道 `ch11` → 引脚 `M13`, 它就是 qCMOS 外触发。确认 XDC 里这根线没被改、`configure_imaging` 的 `trigger_width` (默认 20 us) 足够让相机识别。服务器配置检查里 Channels/触发通道是从 XDC 推断的, 若触发通道推断失败 `run\_server.bat` 会直接报错退出。
- 现象: JTAG 看不到板子** 跑 `build\_and\_program.bat --{-}diagnose` 列出 Vivado 能看到的 hw target/device。看不到就检查 JTAG 线缆、板子供电、以及是否有别的进程占着 `hw_server`。

日常启动顺序 (背下来)

(FPGA 机) 确认 bit 已烧 → `run\_server.bat --{-}check-config` 核对 → `run\_server.bat` 起服务并保持窗口开着; (控制机) `na.connect("remote_template", sequencer=..., open_devices=True)` → `configure_imaging(...)` → `preflight().raise_if_failed()` → 采集。只有改过 RTL/引脚映射时才需要重新 `build\_and\_program.bat`。

13

Pulse API: 脚本控制与延时校准

GUI 之外, 同一套接口可以在 notebook / 脚本里直接驱动 sequencer——**GUI 与 API 走完全相同的路径** (同一个 `sequencer.prepare/fire/set_safe_state`, 同一个 payload 记录), 所以两边永远可以互相同步。核心对象是 `PulseController` (`bind_pulse(sequencer, pulse)` 创建):

Pulse API 一览

```
from Zou_lab_control.neutral_atom.devices.sequencer import
↳ RemoteSequencer, bind_pulse
from Zou_lab_control.neutral_atom import PulseTableState

seq = RemoteSequencer(host="...", port=18861, channels=[f"ch{i:02d}" for i
↳ in range(62)])
pulse = bind_pulse(seq, PulseTableState.load("pulses/T.json")) # 或
↳ pulse.load_pulse(path)

pulse.on_pulse() # 编译 + 上传 + FIRE (=GUI 的 On Pulse)
pulse.off_pulse() # 停止并回安全态 (=GUI 的 Stop Pulse)
pulse.set_channel_delay("ch11", 250) # 设 emCCD 延时 250 ns (链式可连写)
pulse.get_channel_delay("ch11") # 读回 (ns)
pulse.set_scan_table([[10],[20]]) # 上硬件扫描表
pulse.save_pulse("pulses/T2.json") # 存盘 (GUI 可直接 Load)
state = pulse.synced_state() # 读回"设备上实际应用的"PulseTableState
```

13.1 延时校准完整示例

逐通道延时 (事件调度, 上限约 ± 42.9 s, 毫秒级 emCCD 延时直接可用) 的典型校准循环——扫一组延时、每个点 on/off 一次、采一组计数, 最后用前端接口画图:

channel delay calibration

```
import numpy as np
import Zou_lab_control.frontend as zf
```

```
delays_ns = np.linspace(-400, 400, 41)
signal = []
for d in delays_ns:
    pulse.set_channel_delay("ch11", float(d)).on_pulse()
    signal.append(measure())      # 你的采集函数 (相机计数/PMT 等)
    pulse.off_pulse()

fig = zf.plot(delays_ns, np.asarray(signal), kind="1d",
              labels=("delay (ns)", "counts", ""), display=False)
fig.fig.savefig("delay_calibration.png")
best = delays_ns[int(np.argmax(signal))]
pulse.set_channel_delay("ch11", float(best)).on_pulse()  # 应用最优延时
```

Note

脚本改完设备后回到 GUI, 点 **Sync** 即把“设备上实际应用的脉冲”拉回编辑器 (服务端在每次成功 prepare 时记录源 payload, GUI 与 `pulse.synced_state()` 读的是同一份)。反过来, GUI 在 RUNNING 状态下被编辑会让 On Pulse 按钮带上星号 (On Pulse*)、标题栏状态点变橙 (UNSYNCED) ——再点一次即应用。

14

修改指南

14.1 我想改默认时钟

真实硬件默认 50 MHz 是因为当软件/server 假设 100 MHz 时测到的脉冲长了一倍。若换到经过验证的 100 MHz 时钟源，需要同时改 XDC 时钟约束和 `ZLC_PS_CLOCK_HZ=100000000`。所有 `duration/delay/scan` 值都按 `1e9 / clock_hz` 对齐到 tick 网格。

14.2 我想加一条新 channel 或新预设

预设是 `pulses/` 下的 `PulseTableState` JSON，是前端/API 数据，不是硬件工程。新预设可以用 GUI 编辑保存，也可以用 API 构造后 `state.save(path)`。channel 顺序由 server 从 XDC 推断；GUI 隐藏只是视图操作，上传时缺失的 channel 补 off。

14.3 我想加一个扫描参数

不要把扫描网格展开成很多 GUI 列或很多 prepared 表。绑定一个新字段 (`bind_field`)，给 `scan_table` 增加一列，硬件会处理剩下的事。`dac` slot 现在也走硬件无缝扫描（见下一节“扫描 DAC 值”），但不能和**被扫的持续时间**同一次上传（扫 duration 会移动周期起点，让模拟段的绝对 tick 失步——和**延时**扫描则可以同用）。需要扫描模拟 `ramp` 的端点时，按扫描点逐个 `prepare/fire`。

14.4 我想改文档

本手册的正文模板在 `Zou_lab_control/neutral_atom/content/manual_templates/main_manual_zh.texbody`。改完正文后用 `build_main_manual()` 重新生成 PDF（见 `docs/MAINTAINER_NOTES.md` 第 7 节）。

15

常见问题

15.1 脉冲长度是设定的两倍

确认 server、GUI、notebook 都显示 50 MHz / 20 ns。仍按 100 MHz 编译时，设定 1 ms 会在 50 MHz 硬件上输出 2 ms。

15.2 On Pulse 没报错但示波器无信号

按顺序检查：FPGA 已 program 当前 bit/LTX；`-check-config` 指向同一套 bit/LTX/XDC；GUI 打开的是正确预设；示波器接的是 `ch09/ch00/ch03/ch11` 对应的物理输出；脉冲是有限 shot 还是 repeat forever，scope 触发是否设对。

15.3 GUI 只显示少数 channel

这是正常的。GUI 可见列只为降低编辑噪声；上传宽度由 server channel list 决定，缺失 channel 自动补 off。

15.4 扫描不通过

确认每个扫描点的值都是 20 ns 的整数倍；确认时间表达式是 slot 的仿射函数；确认边沿顺序在所有扫描点下不交换。否则拆 chunk 或逐点 prepare。